Experiment 1

Aim: Study and Exploration of Python Libraries Used in Machine Learning Applications.

Learning Objective: To study and understand the use of Python Libraries Pandas and Matplotlib for data manipulation, analysis, and visualization in Machine Learning applications.

Tools: VS Code, I-Python Notebook.

Theory: -

1. Introduction

Machine Learning (ML) is a branch of Artificial Intelligence (AI) that enables computers to learn from data and make predictions without being explicitly programmed. Python is the most widely used programming language for Machine Learning because of its simple syntax, extensive libraries, open-source nature, and strong community support. Libraries such as **NumPy, Pandas, Matplotlib, and Seaborn** provide efficient tools for numerical computation, data manipulation, visualization, and exploratory data analysis (EDA). These libraries form the foundation of most Machine Learning applications by simplifying data preprocessing and analysis before model development.

2. Importance of Python in Machine Learning

Python is preferred for Machine Learning due to its easy-to-learn syntax, platform independence, rich ecosystem of scientific libraries, excellent documentation, and compatibility with frameworks such as Scikit-learn, TensorFlow, PyTorch, and Keras. It enables rapid development, efficient data handling, visualization, and seamless integration of different Machine Learning tasks within a single programming environment.

3. Machine Learning Workflow

A typical Machine Learning project follows several stages: Data Collection → Data Cleaning → Data Preprocessing → Exploratory Data Analysis (EDA) → Feature Engineering → Model Training → Model Evaluation → Prediction and Deployment. NumPy, Pandas, Matplotlib, and Seaborn mainly assist in data preprocessing, analysis, and visualization.

4. NumPy Library

- Introduction
  NumPy (Numerical Python) is the fundamental library for numerical computing in Python. It provides high-performance multidimensional arrays, mathematical functions, linear algebra operations, statistical functions, and random number generation. Many Machine Learning libraries internally rely on NumPy because of its speed and efficient memory management.
- Features
  - High-performance multidimensional arrays.
  - Fast mathematical and statistical computations.
  - Broadcasting and vectorized operations.
  - Linear algebra support.
  - Random number generation.
  - Efficient memory utilization.

- Important Functions
  array(), zeros(), ones(), arange(), linspace(), reshape(), mean(), median(), sum(), max(), min(), sort(), dot(), and transpose().
- Applications
  NumPy is used for numerical computation, matrix operations, vector calculations, dataset creation, feature scaling, statistical analysis, mathematical modeling, and data preprocessing in Machine Learning.
- Advantages
  NumPy offers faster execution than Python lists, consumes less memory, efficiently handles large datasets, supports vectorized operations, and integrates easily with other Machine Learning libraries.

5. Pandas Library

- Introduction
  Pandas is an open-source Python library used for data manipulation and analysis. It introduces two powerful data structures—**Series** and **DataFrame**—that simplify working with structured data stored in CSV, Excel, SQL databases, or JSON files. Pandas is extensively used for cleaning, transforming, organizing, and analyzing datasets before Machine Learning model development.
- Main Data Structures
  - Series: A one-dimensional labeled array capable of storing different types of data.
  - DataFrame: A two-dimensional table consisting of rows and columns used for storing structured datasets.
- Important Functions
  read_csv(), read_excel(), head(), tail(), info(), describe(), shape, columns, drop(), fillna(), isnull(), sort_values(), groupby(), merge(), and concat().
- Applications
  Pandas is widely used for importing datasets, cleaning missing values, removing duplicates, filtering records, transforming data, feature selection, statistical analysis, and preparing datasets for Machine Learning algorithms.
- Advantages
  Pandas simplifies handling of structured data, supports multiple file formats, provides powerful data manipulation functions, integrates seamlessly with NumPy, and significantly reduces preprocessing time.

6. Matplotlib Library

- Introduction
  Matplotlib is a comprehensive visualization library in Python used to create high-quality static, animated, and interactive graphs. It enables users to understand trends, relationships, distributions, and outliers within datasets, making it an essential tool for Exploratory Data Analysis (EDA).
- Features
  - High-quality graphical visualization.
  - Multiple chart types.
  - Extensive customization options.
  - Publication-quality figures.
  - Integration with NumPy and Pandas.

- Common Plot Types
  Line Plot, Bar Chart, Histogram, Scatter Plot, Pie Chart, and Box Plot.
- Important Functions
  plot(), scatter(), bar(), hist(), pie(), xlabel(), ylabel(), title(), legend(), and show().
- Applications
  Matplotlib is used for trend analysis, comparison of datasets, frequency distribution, relationship analysis, performance visualization, and presentation of analytical results.
- Advantages
  It is simple to learn, highly customizable, capable of producing publication-quality graphs, and supports a wide range of visualization techniques.

7. Seaborn Library

- Introduction
  Seaborn is a statistical data visualization library built on top of Matplotlib. It provides attractive default themes and advanced visualization functions for analyzing relationships among variables and performing statistical analysis with minimal code.
- Features
  - Attractive built-in themes.
  - Advanced statistical visualization.
  - Simple and concise syntax.
  - Better integration with Pandas.
  - Supports complex graphical analysis.
- Common Plots
  Heatmap, Pair Plot, Count Plot, Box Plot, Violin Plot, Distribution Plot, Scatter Plot, and Line Plot.
- Important Functions
  heatmap(), pairplot(), countplot(), boxplot(), violinplot(), histplot(), scatterplot(), and lineplot().
- Applications
  Seaborn is commonly used for correlation analysis, statistical visualization, outlier detection, feature relationship analysis, and Exploratory Data Analysis (EDA) before model building.
- Advantages
  Seaborn generates visually appealing statistical plots, requires less coding than Matplotlib for complex visualizations, and offers better default styling for data analysis.

8. Role of These Libraries in Machine Learning

These libraries work together throughout the Machine Learning pipeline. **NumPy** performs mathematical computations, **Pandas** loads and preprocesses datasets, **Matplotlib** visualizes trends and distributions, while **Seaborn** provides advanced statistical analysis and correlation visualization. Together, they prepare high-quality datasets for Machine Learning algorithms such as Linear Regression, Decision Trees, Support Vector Machines, and Neural Networks.

9. Real-World Applications

These libraries are extensively used in stock market prediction, healthcare diagnosis, fraud detection, recommendation systems, weather forecasting, image processing, customer segmentation, sentiment analysis, predictive maintenance, business intelligence, banking, finance, and scientific research.

10. Advantages of Python Libraries for Machine Learning

- Open-source and freely available.
- Easy to learn and implement.
- Fast numerical computation.
- Powerful data manipulation capabilities.
- Excellent visualization support.
- Large community and documentation.
- Compatible with major Machine Learning frameworks.
- Reduces development time and improves productivity.
- Efficient handling of large datasets.
- Suitable for research as well as industrial applications.

11. Limitations

These libraries may consume significant memory while handling extremely large datasets, some visualizations become slower with millions of records, advanced features require practice to master, and Python may execute slower than compiled languages for certain computational tasks, although optimized libraries largely overcome this limitation.

Code and Output:

Code:

```
#1. Importing the libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

print("Libraries imported sucessfully")
```

Output:

Libraries imported successfully

Code:

```
#2. Load the Dataset
df = pd.read_csv("customers-100000.csv")
print("Dataset Imported Sucessfully")
```

Output:

Dataset Imported Sucessfully

Code:

```
#3. Dataset Information
from IPython.display import display
print("First 10 Records")
display(df.head(10))
print("\nShape of Dataset:", df.shape)
print("\nColumn Names:")
display(pd.DataFrame(df.columns, columns=["Columns"]))
print("\nDataset Information:")
df.info()
```

Output:

First 10 Records



	Index	Customer Id	First Name	Last Name	Company	City	Country	Phone 1	Phone 2	Email	Subscription Date	Website
0	1	ffeCAb7Abc80f07	Jared	Jarvis	Sanchez-Fletcher	Hatfieldshire	Eritrea	274.188.8773x41185	001-215-760-4642x969	gabriellehartman@benjamin.com	2021-11-11	https://www.mccarthy.info/
1	2	b687fHC4F1600eC	Marie	Malone	Mckay PLC	Robertsonburgh	Botswana	283-236-9529	(189)129-8356x63741	kstafford@sexton.com	2021-05-14	http://www.reynolds.com/
2	3	9FF9ACbc69dcF9c	Elijah	Barrera	Marks and Sons	Kimbury	Barbados	8252703789	459-916-7241x0909	jeanettecross@brown.com	2021-03-17	https://neal.com/
3	4	b49edDB1295FF6E	Sheryl	Montgomery	Kirby, Vaughn and Sanders	Briannaview	Antarctica (the territory South of 60 deg S)	425.475.3586	(392)819-9063	thomassierra@barrett.com	2020-09-23	https://www.powell-bryan.com/
4	5	3dcCbFEB17CCf2E	Jeremy	Houston	Lester-Manning	South Brianna	Micronesia	+1-223-666-5313x4530	252-488-3850x692	rubenwatkins@jacobs-wallace.info	2020-09-18	https://www.camillo.com/
5	6	75bEe1B33E517AF	Frank	Villarreal	Craig, Dennis and Maddox	Reidtown	Panama	+1-165-340-7100x44099	(847)867-2795x043	gmurillo@perez.com	2022-01-13	https://patel.org/
6	7	CBBb29B6Fbbbe55	Rickey	Fox	Benson, Gray and Fitzpatrick	Mcintoshberg	United States of America	426.446.3838	437.476.4705	melissabenitez@le.info	2021-11-05	http://www.irwin-donaldson.com/
7	8	56b3cEA1E6A49F1	Barry	Ware	Glenn Ltd	Lake Pedrofort	American Samoa	101-419-0208x043	931.542.3115x98926	brent60@lopez.com	2022-05-12	https://www.bean.biz/
8	9	f9443t5f6AF0Da0	Carl	Mcdowell	Davila-Leach	Petersbury	Saint Vincent and the Grenadines	+1-740-392-0511	+1-218-891-5403x7270	toddharris@lowery-rosario.biz	2022-05-28	https://www.burnett.com/
9	10	C99b63E0d92C565	Pam	Dalton	Mckenzie Ltd	Hillside	Tanzania	0389689232	344-952-1181x6911	carolbanks@vang-stafford.com	2022-03-04	http://www.maxwell-bradford.com/

Shape of Dataset: (100000, 12)

Column Names:

	Columns
0	Index
1	Customer Id
2	First Name
3	Last Name
4	Company
5	City
6	Country
7	Phone 1
8	Phone 2
9	Email
10	Subscription Date
11	Website

Dataset Information:

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 12 columns):

#	Column	Non-Null Count	Dtype
0	Index	100000 non-null	int64
1	Customer Id	100000 non-null	object
2	First Name	100000 non-null	object
3	Last Name	100000 non-null	object
4	Company	100000 non-null	object
5	City	100000 non-null	object
6	Country	100000 non-null	object
7	Phone 1	100000 non-null	object
8	Phone 2	100000 non-null	object
9	Email	100000 non-null	object

10	Subscription Date	100000 non-null	object
11	Website	100000 non-null	object

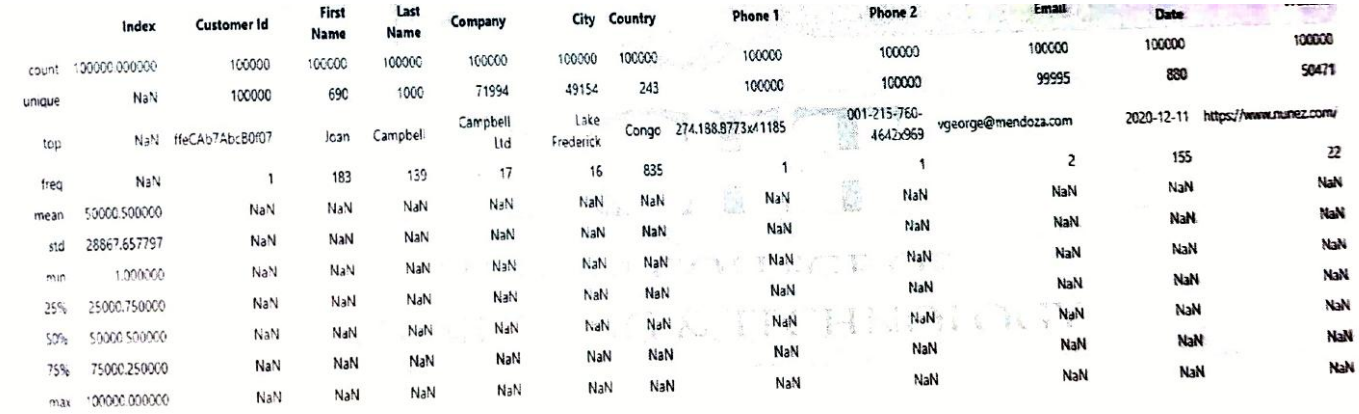
dtypes: int64(1), object(11)
memory usage: 9.2+ MB

Code:

```
# 4. Statistical Summary

df.describe(include='all')
```

Output:



	Index	Customer Id	First Name	Last Name	Company	City	Country	Phone 1	Phone 2	Email	Date	
count	100000.000000	100000	100000	100000	100000	100000	100000	100000	100000	100000	100000	100000
unique	NaN	100000	690	1000	71994	49154	243	100000	100000	99995	880	50471
top	NaN	ffeCAb7AbcB0f07	Joan	Campbell	Campbell Ltd	Lake Frederick	Congo	274.188.8773x41185	001-215-760-4642x969	vgeorge@mendoza.com	2020-12-11	https://www.nunez.com/
freq	NaN	1	183	139	17	16	835	1	1	2	155	22
mean	50000.500000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
std	28867.657797	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
min	1.000000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
25%	25000.750000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
50%	50000.500000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
75%	75000.250000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
max	100000.000000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

Code:

```
#5. Check Missing Values
print("Missing Values:\n")
print(df.isnull().sum())
```

Output:

Missing Values:

Index	0
Customer Id	0
First Name	0
Last Name	0
Company	0
City	0
Country	0
Phone 1	0
Phone 2	0
Email	0
Subscription Date	0
Website	0

dtype: int64

Code:

```
#6. Display Random Sample
df.sample(10)
```

TCET
THAKUR COLLEGE OF
ENGINEERING & TECHNOLOGY

Output:



	Index	Customer Id	First Name	Last Name	Company	City	Country	Phone 1	Phone 2			
36024	36025	3F72b9D6a3f86da	Yolanda	Eaton	Mcgrath-Gonzales	North Albert	Azerbaijan	001-492-792-6089	841-024-6258x950	patricklarry@mercado.com	2021-01-04	http://parrish-doyle.com/
41368	41369	78BcaDB25A4B80A	Jean	Kent	Moyer-Jones	Lake Tonya	Bulgaria	787-991-2087	136.263.0144	jacksonshelby@gibbs.org	2020-03-01	https://rice-mueller.com/
49913	49914	DfB4B5aa1FfbFab	Shannon	Chase	Richards-Day	Lake Nicholaston	Congo	219-538-7084	001-286-386-0259x68564	bondtracy@chaney-morton.org	2021-10-04	https://www.russell.com/
33676	33677	590aD8bAB3426bf	Alicia	Dorsey	Mclean Ltd	Davisview	El Salvador	001-522-704-6182x119	(132)076-3849x252	matthewhaynes@becker.info	2022-02-21	https://www.banks.net/
40486	40487	4cffD103CAC8B6a	Barry	Daniel	Atkinson and Sons	Port Glenbury	Vietnam	+1-553-697-4647x3448	(663)181-1513x607	frederickbailey@chase.biz	2022-05-21	https://barrera-cooley.com/
47831	47832	2Dd45173BecEDa5	Courtney	Estes	Chambers, Crawford and Berry	North Jackport	Austria	847.122.1849x742	627-751-1501	alex30@melton.com	2020-06-13	http://www.hayes-gomez.com/
62982	62983	10D6520751e3f8f	Jake	Ochoa	Aguilar PLC	Patrickchester	Paraguay	001-294-351-4186x0759	+1-460-158-6288	brianna86@brock.com	2020-08-25	https://pope.net/
93206	93207	5Be551f9144cD2b	Sergio	Mercer	Mullen-Miller	West Jayfurt	Pakistan	440.088.7062x19690	+1-599-136-5819	dannysullivan@estes.org	2022-05-23	https://dalton-ball.com/
80349	80350	BE2A9575f0Be7F8	Lance	Andrade	Kim, Burgess and Weiss	Rothshire	Chile	001-133-519-9506x0173	624-720-6990	erik22@hatfield-shea.net	2020-02-09	https://www.house.com/
34051	34052	BC102aAa72Fb0fc	Shane	Mathews	Ortega, Vazquez and Mills	Port Wendy	Andorra	843-096-3755x82150	+1-969-020-1217x438	hollypetersen@bentley-montgomery.com	2020-04-10	https://www.phillips.com/

Code:

```
#7. Data Selection - Displaying only selected columns
df[['First Name','Last Name','Country']].head(10)
```

Output:

	First Name	Last Name	Country
0	Jared	Jarvis	Eritrea
1	Marie	Malone	Botswana
2	Elijah	Barrera	Barbados
3	Sheryl	Montgomery	Antarctica (the territory South of 60 deg S)
4	Jeremy	Houston	Micronesia
5	Frank	Villarreal	Panama
6	Rickey	Fox	United States of America
7	Barry	Ware	American Samoa
8	Carl	Mcdowell	Saint Vincent and the Grenadines
9	Pam	Dalton	Tanzania

Code:

```
#8. Performing Numerical Operations
#Count Customers By country
cust_by_country = df['Country'].value_counts()
print(cust_by_country)
```

Output:

```
Country
Congo                 835
Korea                 820
Saudi Arabia          463
Pitcairn Islands      456
Saint Martin          453
...
Moldova               371
Jordan                365
Saint Lucia           365
```

Slovenia	361
Greece	359

Code:

```
# Displaying Top 10 Countries
top10 = df['Country'].value_counts().head(10)
print(top10)
```

Output:

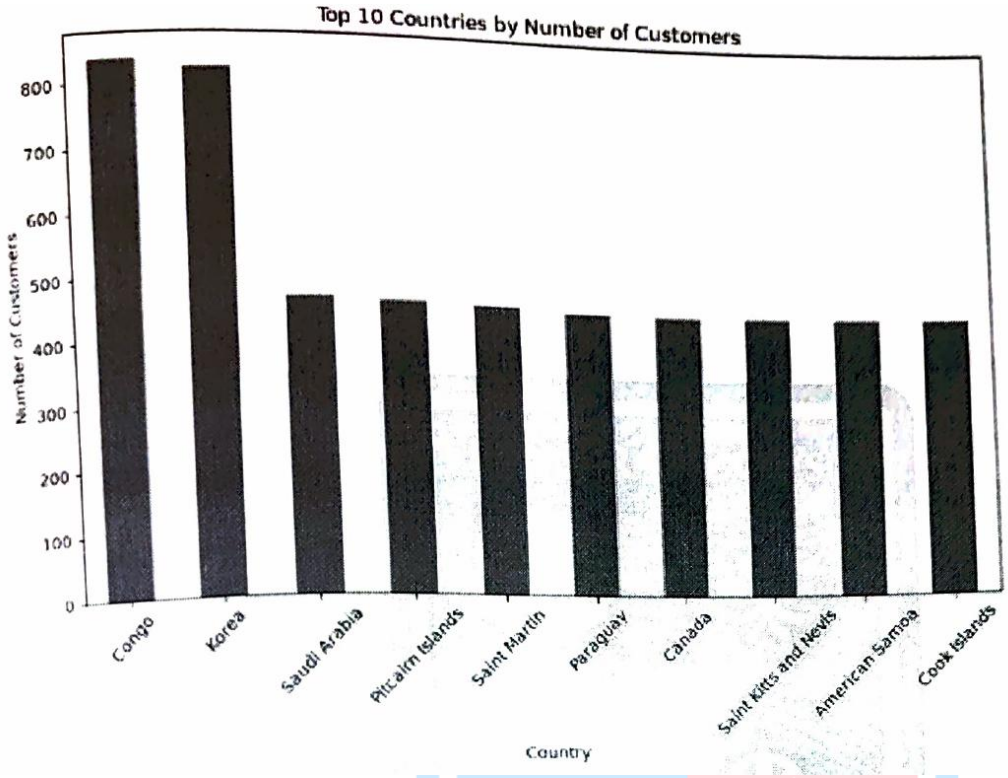
```
Country
Congo                 835
Korea                 820
Saudi Arabia          463
Pitcairn Islands      456
Saint Martin          453
Paraguay              445
Canada                444
Saint Kitts and Nevis 443
American Samoa        443
Cook Islands          441
Name: count, dtype: int64
```

Code:

```
# Performing matplotlib operations
# Bar Chart
plt.figure(figsize=(10,6))
top10.plot(kind='bar')
plt.title("Top 10 Countries by Number of Customers")
plt.xlabel("Country")
plt.ylabel("Number of Customers")
plt.xticks(rotation=45)
plt.show()
```

TCET
THAKUR COLLEGE OF
ENGINEERING & TECHNOLOGY

Output:



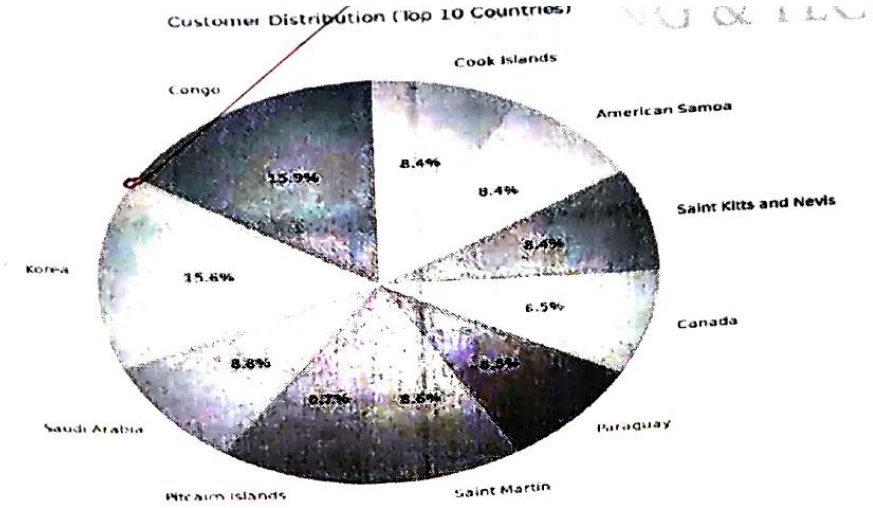
Code:

```
# Pie Chart
plt.figure(figsize=(8,8))
top10.plot(
    kind='pie',
    autopct='%1.1f%%',
    startangle=90
)
plt.title("Customer Distribution (Top 10 Countries)")
plt.ylabel("")

plt.show()
```

TCET
THAKUR COLLEGE OF
ENGINEERING & TECHNOLOGY

Output:



TCET
THAKUR COLLEGE OF
ENGINEERING & TECHNOLOGY

Learning Outcomes: The student should have the ability to:

LO 1.1: Outcomes:1Understand the role of Pandas in data manipulation and analysis.

LO 1.2: Learn to visualize data using Matplotlib

Course Outcomes: Understand basics of ML and Python.

Conclusion:

For Faculty Use: -

Correction Parameters	Formative Assessment [40%]	Timely completion of Practical [40%]	Attendance / Learning Attitude [20%]	
Marks Obtained				

Experiment 2

Aim: Implementation of Simple, Multiple and Logistic Regression and model performance assessment.

Learning Objective: To understand the concepts and applications of Simple Linear Regression, Multiple Linear Regression, and Logistic Regression.

Tools: VS Code, I-Python Notebook.

Theory: -

Introduction to Regression

Regression is one of the most important supervised machine learning techniques used to predict numerical or categorical outcomes based on historical data. It identifies the relationship between dependent (target) variables and one or more independent (input) variables. Regression analysis helps organizations make informed decisions by forecasting future trends and understanding the influence of different factors on an outcome. It is widely used in finance, healthcare, marketing, education, manufacturing, and scientific research.

Machine learning regression models learn patterns from training data and use those learned relationships to predict outputs for unseen data. Depending on the type of target variable, different regression techniques are applied. When the target variable is continuous, Linear Regression is used, whereas Logistic Regression is applied when the target variable is categorical or binary.

Simple Linear Regression

Simple Linear Regression (SLR) is a supervised learning algorithm used to model the relationship between one independent variable and one dependent variable. It assumes that the relationship between the variables is linear and attempts to fit the best straight line through the data points.

The mathematical equation of Simple Linear Regression is:

$Y = \beta_0 + \beta_1 X$

Where:

- Y = Dependent Variable
- X = Independent Variable
- β_0 = Intercept
- β_1 = Slope (Coefficient)

The model determines the best values of the slope and intercept by minimizing the error between the predicted and actual values using the Least Squares Method.

Working Process

1. Collect training data.
2. Define independent and dependent variables.
3. Train the Linear Regression model.
4. Calculate the best-fit regression line.
5. Predict output values.
6. Evaluate model performance using regression metrics.

Applications

- House price prediction
- Sales forecasting

- Weather forecasting
- Population prediction
- Business revenue estimation

Multiple Linear Regression

Multiple Linear Regression (MLR) extends Simple Linear Regression by considering two or more independent variables to predict a single dependent variable. It is useful when the target variable depends on multiple factors.

The mathematical equation is:

$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + ... + \beta_n X_n$

where:

- Y = Dependent variable
- $X_1, X_2, ... X_n$ = Independent variables
- β_0 = Intercept
- $\beta_1, \beta_2, ... \beta_n$ = Regression coefficients

Each coefficient represents the effect of its corresponding independent variable while keeping other variables constant.

Working Process

1. Collect dataset with multiple features.
2. Select independent and dependent variables.
3. Train the Multiple Linear Regression model.
4. Estimate regression coefficients.
5. Predict target values.
6. Evaluate prediction accuracy.

Applications

- Student performance prediction
- Stock market analysis
- Medical diagnosis
- Demand forecasting
- Business analytics

Logistic Regression

Logistic Regression is a supervised machine learning classification algorithm used when the dependent variable is categorical. Although its name contains "Regression," it is primarily used for binary classification problems such as Yes/No, Pass/Fail, Disease/No Disease, Spam/Not Spam, etc.

Instead of predicting continuous values, Logistic Regression predicts the probability of belonging to a particular class using the Sigmoid (Logistic) Function.

The sigmoid function is:

$P = 1 / (1 + e^{-Z})$

where

$Z = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + ... + \beta_n X_n$

The sigmoid function converts any real number into a probability between 0 and 1. Based on a threshold (usually 0.5), the probability is converted into a class label.

Working Process

1. Prepare labeled training data.
2. Train the Logistic Regression model.
3. Calculate probabilities using the sigmoid function.
4. Apply threshold classification.
5. Predict class labels.
6. Evaluate model performance.

Applications

- Disease prediction
- Email spam detection
- Customer churn prediction
- Fraud detection
- Credit approval systems

Model Performance Assessment

After training a machine learning model, its performance must be evaluated using suitable evaluation metrics.

Performance Metrics for Linear Regression

1. Mean Absolute Error (MAE): Measures the average absolute difference between actual and predicted values. Lower MAE indicates better performance.

2. Mean Squared Error (MSE): Measures the average squared error between actual and predicted values. It penalizes larger errors more heavily.

3. Root Mean Squared Error (RMSE): The square root of MSE. It represents prediction error in the same units as the target variable.

4. R^2 Score (Coefficient of Determination): Measures how well the regression model explains the variance in the data. An R^2 value closer to 1 indicates better model performance.

Performance Metrics for Logistic Regression

Accuracy: Measures the percentage of correctly classified instances.

Precision: Indicates how many predicted positive cases are actually positive.

Recall (Sensitivity): Measures how many actual positive cases are correctly identified.

F1-Score: The harmonic mean of Precision and Recall, providing a balanced evaluation.

Confusion Matrix

A table that summarizes classification performance using:

- True Positives (TP)
- True Negatives (TN)
- False Positives (FP)
- False Negatives (FN)

These values help understand where the classifier is making correct and incorrect predictions.

Code and Output:

Code:

```
# Importing the Libraries

import pandas as pd
```

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score,
    accuracy_score,
    confusion_matrix,
    classification_report
)

#Importing the Dataset
df = pd.read_csv("sales_data_sample.csv", encoding="latin1")
```

Output:

Code Executed Successfully

Code:

```
#Simple Linear Regression
X = df[['PRICEEACH']]
y = df['SALES']
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
simple_model = LinearRegression()
simple_model.fit(X_train, y_train)
simple_prediction = simple_model.predict(X_test)
print("Simple Linear Regression Results\n")
print("MAE :", mean_absolute_error(y_test, simple_prediction))
print("MSE :", mean_squared_error(y_test, simple_prediction))
print("RMSE :", np.sqrt(mean_squared_error(y_test, simple_prediction)))
print("R² Score :", r2_score(y_test, simple_prediction))
```

Output:

Simple Linear Regression Results

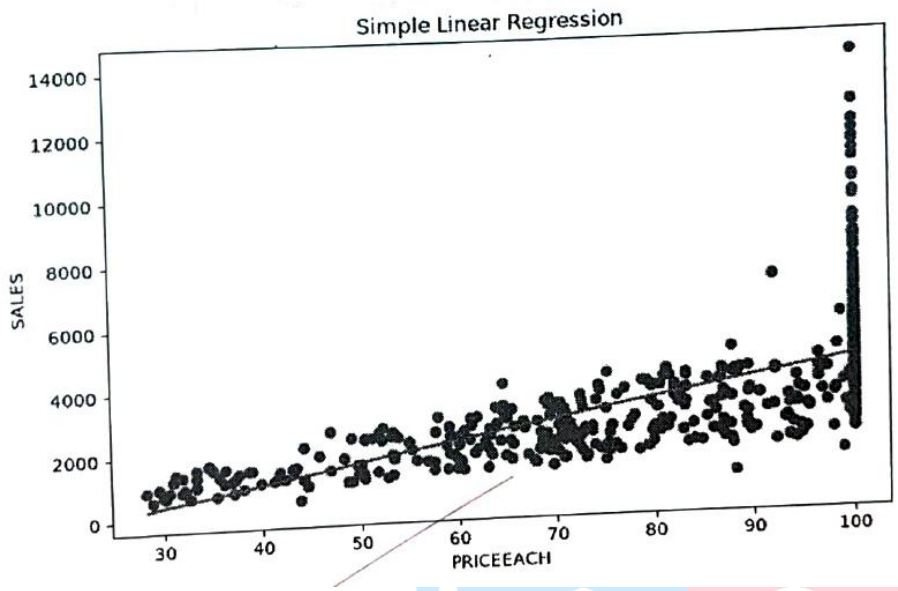
MAE : 1114.7201701938488
MSE : 2606141.4195963386
RMSE : 1614.354799787314
R² Score : 0.40293081286558696

Code:

```
plt.figure(figsize=(8,5))
plt.scatter(X_test, y_test)
```

```
plt.plot(X_test, simple_prediction, color='red')
plt.title("Simple Linear Regression")
plt.xlabel("PRICEEACH")
plt.ylabel("SALES")
plt.show()
```

Output:



Code:

```
#Multiple Regression - Predict Sales Using Multiple Features
X = df[['PRICEEACH',
        'QUANTITYORDERED',
        'MSRP']]
y = df['SALES']
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
multiple_model = LinearRegression()
multiple_model.fit(X_train, y_train)
multiple_prediction = multiple_model.predict(X_test)
print("Multiple Linear Regression Results\n")
print("MAE :", mean_absolute_error(y_test, multiple_prediction))
print("MSE :", mean_squared_error(y_test, multiple_prediction))
print("RMSE :", np.sqrt(mean_squared_error(y_test, multiple_prediction))
print("R² Score :", r2_score(y_test, multiple_prediction)))
```

Output:

```
Multiple Linear Regression Results

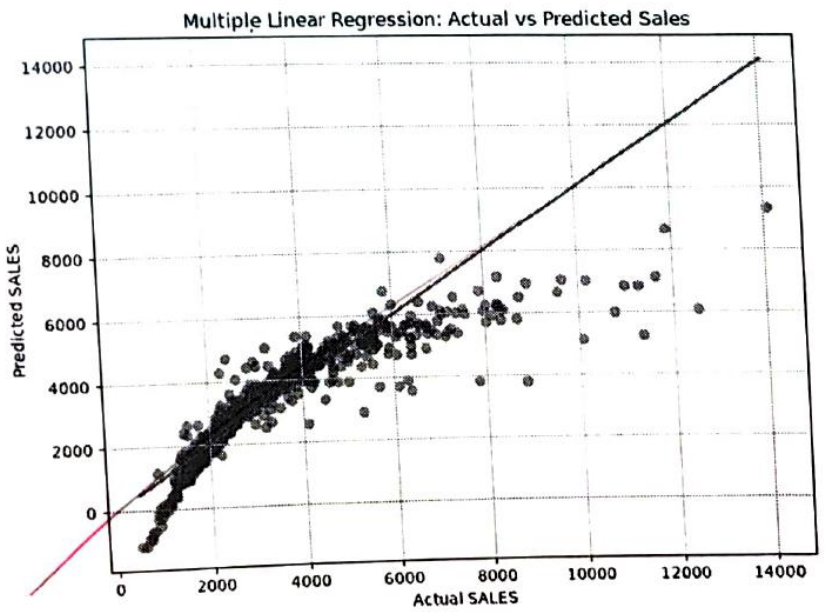
MAE : 653.0680867292306
MSE : 1045246.550427748
```

RMSE : 1022.3729996570469
R^2 Score : 0.7605331377928032

Code:

```
plt.figure(figsize=(8,6))
plt.scatter(y_test, multiple_prediction, alpha=0.6)
# Perfect prediction line
plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    color='red',
    linewidth=2
)
plt.xlabel("Actual SALES")
plt.ylabel("Predicted SALES")
plt.title("Multiple Linear Regression: Actual vs Predicted Sales")
plt.grid(True)
plt.show()
```

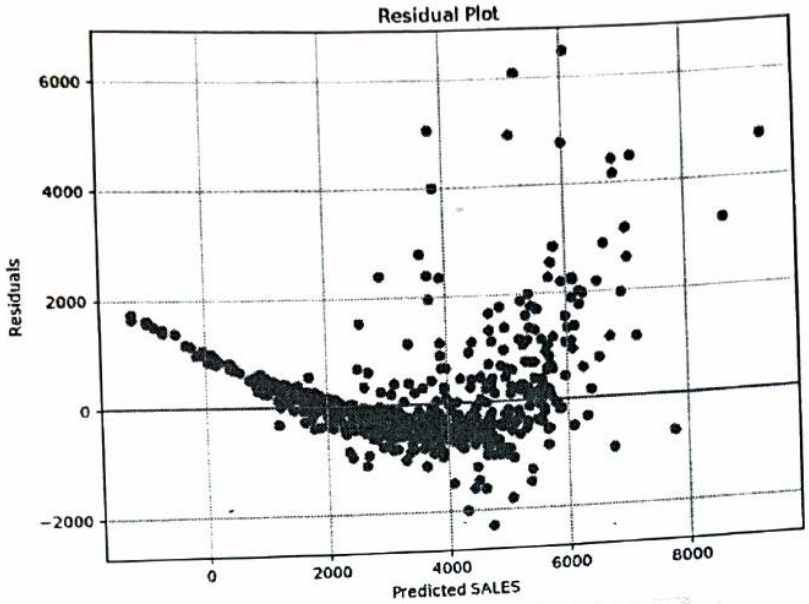
Output:



Code:

```
residuals = y_test - multiple_prediction
plt.figure(figsize=(8,6))
plt.scatter(multiple_prediction, residuals)
plt.axhline(y=0, color='red')
plt.xlabel("Predicted SALES")
plt.ylabel("Residuals")
plt.title("Residual Plot")
plt.grid(True)
plt.show()
```

Output:



Code:

```
#Logistic Regression Predict Deal Size

encoder = LabelEncoder()
df['DEALSIZE'] = encoder.fit_transform(df['DEALSIZE'])
X = df[['PRICEEACH',
        'QUANTITYORDERED',
        'MSRP']]
y = df['DEALSIZE']
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
logistic_model = LogisticRegression(max_iter=1000)
logistic_model.fit(X_train, y_train)
logistic_prediction = logistic_model.predict(X_test)
print("Accuracy :", accuracy_score(y_test, logistic_prediction))
cm = confusion_matrix(y_test, logistic_prediction)
print("Confusion Matrix\n")
print(cm)
print(classification_report(y_test, logistic_prediction))
```

Output:

```
Accuracy : 0.8938053097345132
Confusion Matrix

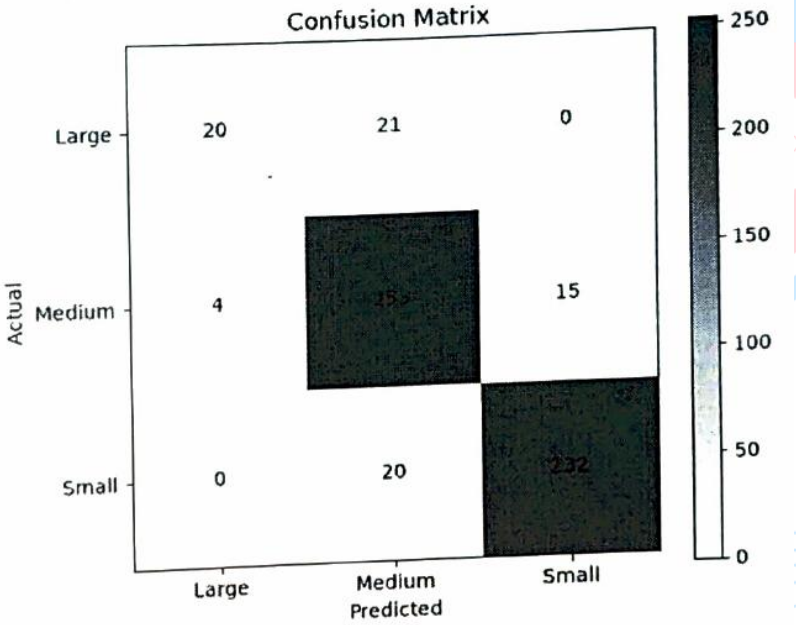
[[ 20  21   0]
 [  4 253  15]
 [  0  20 232]]
```

Code:

```
plt.figure(figsize=(6,5))

plt.imshow(cm, cmap='Blues')
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.colorbar()
labels = encoder.classes_
plt.xticks(range(len(labels)), labels)
plt.yticks(range(len(labels)), labels)
for i in range(cm.shape[0]):
    for j in range(cm.shape[1]):
        plt.text(j, i, cm[i, j],
                 ha='center',
                 va='center',
                 color='black')
plt.show()
```

Output:



Code:

```
summary = pd.DataFrame({

    "Model": [
        "Simple Linear Regression",
        "Multiple Linear Regression",
        "Logistic Regression"
    ],
    "Performance": [
        f"R² = {r2_score(y_test, simple_prediction):.3f}",
        f"R² = {r2_score(y_test, multiple_prediction):.3f}",
        f"Accuracy = {accuracy_score(y_test, logistic_prediction):.3f}"
```

```
    ]
})
display(summary)
```

Output:

	Model	Performance
0	Simple Linear Regression	R^2 = -36562402.031
1	Multiple Linear Regression	R^2 = -40545291.473
2	Logistic Regression	Accuracy = 0.894

TCET
THAKUR COLLEGE OF
ENGINEERING & TECHNOLOGY

Learning Outcomes: The student should have the ability to:

LO 2.1: Apply simple and multiple linear regression models using Python for predicting continuous variables.

LO 2.2: Assess model performance using evaluation metrics such as R^2 Score, Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE) for linear regression, and Accuracy, Precision, Recall, F1-Score, ROC-AUC, and Confusion Matrix for logistic regression.

Course Outcomes: Develop and evaluate regression models for predictive analysis.

Conclusion:

For Faculty Use: -

Correction Parameters	Formative Assessment [40%]	Timely completion of Practical [40%]	Attendance / Learning Attitude [20%]	
Marks Obtained				

Experiment 3

Aim: Build and evaluate decision tree classifiers for classification tasks using scikit learn

Learning Objective: To understand the concept of Decision Tree algorithm and apply Decision Tree classifier on actual dataset to do the classification.

Tools: VS Code, I-Python Notebook.

Theory: -

1. Introduction

Decision Tree is one of the most widely used supervised machine learning algorithms for solving classification and regression problems. It works by recursively dividing the dataset into smaller subsets based on the values of input features. The resulting model resembles a tree-like structure where each internal node represents a decision based on a feature, each branch represents the outcome of that decision, and each leaf node represents the final predicted class. Decision Trees are popular because they are simple to understand, easy to visualize, and require minimal data preprocessing compared to many other machine learning algorithms.

In this experiment, the Decision Tree Classifier is implemented using the **Scikit-learn** library in Python to classify records from the given sales dataset. The classifier learns patterns from historical data and predicts the class label (such as order status) for unseen records. The performance of the model is then evaluated using various classification metrics such as accuracy, confusion matrix, precision, recall, and F1-score.

2. Supervised Learning

Decision Tree Classification belongs to the category of **Supervised Machine Learning**. In supervised learning, the model is trained using a dataset that contains both input features (independent variables) and the corresponding target labels (dependent variable). During training, the algorithm learns the relationship between the input features and the target variable. Once trained, the model can accurately predict the class of new unseen data.

The objective of supervised learning is to minimize prediction errors by learning meaningful patterns from the training dataset.

3. Decision Tree Algorithm

A Decision Tree is a flowchart-like structure where the classification process begins from the **root node** and continues through several decision nodes until it reaches a **leaf node**, which represents the predicted class.

The algorithm repeatedly selects the best feature for splitting the dataset based on an impurity measure such as **Gini Index** or **Entropy (Information Gain)**. The process continues recursively until one of the stopping conditions is met, such as reaching the maximum tree depth or when all samples belong to the same class.

Components of a Decision Tree

- **Root Node:** Represents the entire dataset and the first decision.
- **Decision Node:** A node where the dataset is divided based on a feature.
- **Branch:** Represents the outcome of a decision.
- **Leaf Node:** Represents the final predicted class.
- **Parent Node:** A node that splits into child nodes.
- **Child Node:** A node generated after splitting the parent node.

4. Working of Decision Tree

The Decision Tree algorithm follows the steps below:

1. Load the dataset.
2. Select the target variable for classification.
3. Choose the best feature for splitting using Gini Index or Entropy.
4. Split the dataset into smaller subsets.
5. Repeat the splitting process recursively.
6. Stop splitting when the stopping criteria are satisfied.
7. Predict the class of new records using the generated tree.

This hierarchical decision-making process enables the model to classify data accurately while maintaining interpretability.

5. Splitting Criteria

The quality of each split is determined using an impurity measure.

(a) Gini Index

The Gini Index measures how often a randomly chosen element would be incorrectly classified if it were randomly labeled according to the distribution of labels.

Characteristics:

- Lower Gini value indicates better purity.
- Gini value ranges from 0 to 1.
- A value of 0 indicates a perfectly pure node.

(b) Entropy and Information Gain

Entropy measures the amount of randomness or uncertainty present in the dataset. The Decision Tree selects the feature that provides the maximum Information Gain, which means the greatest reduction in uncertainty after splitting.

Characteristics:

- Higher entropy indicates greater disorder.
- Lower entropy indicates purer nodes.
- Information Gain helps in selecting the optimal feature for splitting.

6. Scikit-learn Library

Scikit-learn is a powerful open-source Python library used for machine learning and data mining. It provides efficient implementations of various machine learning algorithms including Decision Trees, Random Forest, Support Vector Machine, Logistic Regression, and many others.

In this experiment, Scikit-learn provides:

- Dataset splitting using train_test_split()
- Label encoding using LabelEncoder
- Model creation using DecisionTreeClassifier
- Prediction using predict()
- Performance evaluation using accuracy_score(), confusion_matrix(), and classification_report()
- Tree visualization using plot_tree()

7. Dataset Used

The experiment uses the **Sales Data Sample** dataset, which contains customer order information including product details, order quantity, sales amount, product line, order month, year, and order status.

The target variable selected for classification is:

- **STATUS**

The remaining relevant attributes are used as input features to train the Decision Tree model. Before training, categorical features are converted into numerical form using **Label Encoding**, and unnecessary columns are removed to improve model performance.

8. Model Training and Testing

The dataset is divided into two parts:

- **Training Set (80%)** – Used to train the Decision Tree model.
- **Testing Set (20%)** – Used to evaluate the model's performance.

The classifier learns decision rules from the training data and predicts the class labels for the testing dataset.

9. Model Evaluation

After training, the model performance is evaluated using standard classification metrics.

Accuracy

Accuracy measures the percentage of correctly classified instances.

- Indicates the overall performance of the classifier.
- Higher accuracy represents better prediction capability.

Confusion Matrix

The confusion matrix compares the actual class labels with the predicted labels.
It provides information about:

- Correct predictions
- Incorrect predictions
- Misclassified instances

Classification Report

The classification report summarizes the model performance using:

- **Precision:** Measures how many predicted positive instances are actually correct.
- **Recall:** Measures how many actual positive instances are correctly identified.
- **F1-Score:** Harmonic mean of Precision and Recall.
- **Support:** Number of actual occurrences of each class.

These metrics provide a comprehensive evaluation of the Decision Tree classifier.

10. Advantages of Decision Tree

- Simple and easy to understand.
- Easy to visualize.
- Requires little data preprocessing.
- Handles both numerical and categorical data.
- Performs feature selection automatically.
- Works well for multi-class classification problems.
- Easy to interpret and explain.

11. Limitations of Decision Tree

- Can easily overfit the training dataset.
- Sensitive to small variations in data.
- Large trees become complex and difficult to interpret.
- Performance may decrease on noisy datasets.
- Less stable compared to ensemble methods like Random Forest.

12. Applications of Decision Tree

Decision Tree Classification is widely used in various real-world applications, including:

- Customer purchase prediction
- Sales forecasting and order classification
- Medical diagnosis
- Fraud detection
- Credit risk analysis
- Email spam detection
- Customer churn prediction
- Loan approval systems
- Marketing analytics
- Recommendation systems

Code and Output:

Code:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Load Dataset
df = pd.read_csv("sales_data_sample.csv", encoding="latin1")

# Drop unnecessary columns
drop_cols = [
    "ORDERNUMBER",
    "ORDERDATE",
    "PHONE",
    "ADDRESSLINE1",
    "ADDRESSLINE2",
    "CITY",
    "STATE",
    "POSTALCODE",
    "COUNTRY",
    "TERRITORY",
    "CONTACTFIRSTNAME",
    "CONTACTLASTNAME",
```

```
    "CUSTOMERNAME"
]
df.drop(columns=drop_cols, inplace=True, errors="ignore")

# Fill Missing Values
df.fillna("Unknown", inplace=True)

# Encode Categorical Columns
le = LabelEncoder()

for col in df.columns:
    if df[col].dtype == "object":
        df[col] = le.fit_transform(df[col].astype(str))

# Features and Target
X = df.drop("STATUS", axis=1)
y = df["STATUS"]

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

# Decision Tree Model
dt = DecisionTreeClassifier(
    criterion="entropy",     # or "gini"
    max_depth=3,             # Keeps tree clean
    min_samples_split=20,
    min_samples_leaf=10,
    random_state=42
)
dt.fit(X_train, y_train)

# Prediction
y_pred = dt.predict(X_test)

# Evaluation
print("Accuracy :", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report")
print(classification_report(y_test, y_pred))

# Decision Tree Plot
plt.figure(figsize=(24,12), dpi=200)
plot_tree(
```

```
    dt,
    feature_names=X.columns,
    class_names=[str(i) for i in dt.classes_],
    filled=True,
    rounded=True,
    fontsize=9,
    max_depth=2         # Shows only first 3 levels
)
plt.tight_layout()
plt.show()
```

Output:

Accuracy : 0.9274336283185841

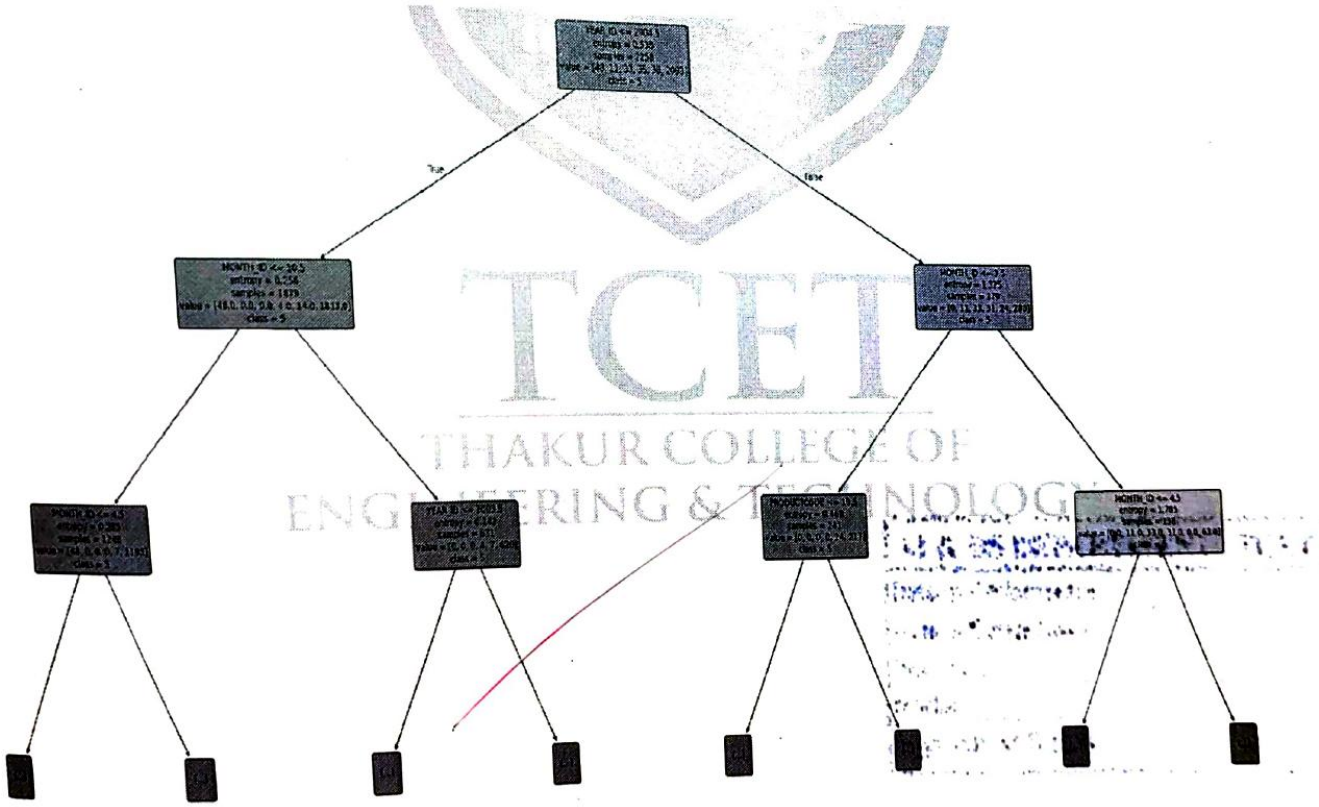
Confusion Matrix

```
[[  0   0   0   0   0  12]
 [  0   0   0   0   0   3]
 [  0   0   0   0   0   8]
 [  0   0   0   0   0   9]
 [  0   0   0   0   0   9]
 [  0   0   0   0   0 524]]
```

Classification Report

	precision	recall	f1-score	support
0	0.00	0.00	0.00	12
1	0.00	0.00	0.00	3
2	0.00	0.00	0.00	8
3	0.00	0.00	0.00	9
4	0.00	0.00	0.00	9
5	0.93	1.00	0.96	524
accuracy			0.93	565
macro avg	0.15	0.17	0.16	565
weighted avg	0.86	0.93	0.89	565

THAKUR ZCT'S
TCET
THAKUR COLLEGE OF ENGINEERING & TECHNOLOGY



TCET
THAKUR COLLEGE OF
ENGINEERING & TECHNOLOGY

Learning Outcomes: The student should have the ability to:

LO 3.1: Understand the working principles of the Decision Tree algorithm and its application in supervised classification tasks.

LO 3.2: Build and train a Decision Tree Classifier using the Scikit-learn library by preprocessing the dataset, splitting it into training and testing sets, and fitting the model.

LO 3.3: Evaluate the performance of the Decision Tree Classifier using metrics such as Accuracy, Confusion Matrix, Precision, Recall, and F1-Score, and interpret the classification results.

Course Outcomes: Apply supervised machine learning techniques using Scikit-learn to build, evaluate, and interpret Decision Tree classification models for solving real-world data classification problems.

Conclusion:

For Faculty Use: -

Correction Parameters	Formative Assessment [40%]	Timely completion of Practical [40%]	Attendance / Learning Attitude [20%]	
Marks Obtained				

Experiment 4

Aim: Implementation of Support Vector Machine (SVM) for Classification, Regression and Outlier Detection

Learning Objective: To understand the working principle of Support Vector Machine (SVM), implement SVM algorithms for classification, regression, and outlier detection, and evaluate their performance using appropriate machine learning techniques and evaluation metrics.

Tools: VS Code, I-Python Notebook.

Theory: -

1. Introduction

Support Vector Machine (SVM) is a supervised machine learning algorithm used for both classification and regression tasks. It works by finding an optimal decision boundary, known as a **hyperplane**, that maximizes the margin between different classes. The support vectors are the data points closest to the hyperplane, and they play a crucial role in defining the decision boundary. SVM is highly effective for both linear and non-linear datasets and is widely used due to its high accuracy and robustness.

2. Support Vector Machine (SVM)

Support Vector Machine is a powerful learning algorithm that constructs an optimal separating hyperplane between different classes. Unlike many traditional classifiers, SVM focuses on maximizing the distance (margin) between the nearest data points of different classes, thereby improving the model's ability to generalize to unseen data.

The main components of SVM include:

- **Hyperplane:** A decision boundary that separates different classes.
- **Support Vectors:** Data points closest to the hyperplane that determine its position.
- **Margin:** The maximum distance between the hyperplane and the nearest support vectors.
- **Kernel Function:** Maps data into a higher-dimensional space when the data is not linearly separable.

3. SVM for Classification

In classification problems, SVM separates data into different classes by identifying the optimal hyperplane that maximizes the margin. For non-linearly separable data, kernel functions transform the data into higher dimensions where a linear separation becomes possible.

The classification process includes:

- Preparing and preprocessing the dataset.
- Selecting relevant features.
- Scaling numerical features.
- Training the SVM classifier.
- Predicting class labels for test data.
- Evaluating model performance using standard metrics.

4. SVM for Regression (SVR)

Support Vector Regression (SVR) extends the SVM concept to regression problems by predicting continuous numerical values instead of class labels. Instead of minimizing prediction error directly, SVR attempts to fit the best possible function within a specified error tolerance (ε), while maintaining maximum margin.

SVR is commonly used for:

- Sales prediction.
- Stock price forecasting.
- Demand forecasting.
- Weather prediction.

5. SVM for Outlier Detection

Support Vector Machine can also detect anomalies using **One-Class SVM**. The algorithm learns the normal behavior of the training data and identifies observations that significantly differ from the learned pattern as outliers.

Applications include:

- Fraud detection.
- Network intrusion detection.
- Fault detection in industrial systems.
- Medical anomaly detection.

6. Kernel Functions

Kernel functions enable SVM to solve non-linear problems by transforming data into higher-dimensional feature spaces.

Common kernel functions include:

- **Linear Kernel:** Suitable for linearly separable data.
- **Polynomial Kernel:** Captures polynomial relationships between features.
- **Radial Basis Function (RBF) Kernel:** Most widely used kernel for handling complex and non-linear datasets.
- **Sigmoid Kernel:** Similar to neural network activation functions for certain applications.

7. Applications of Support Vector Machine

Support Vector Machine is widely used in various domains, including:

- Email spam classification.
- Image and face recognition.
- Medical disease diagnosis.
- Handwritten character recognition.
- Customer churn prediction.
- Financial risk analysis.

- Text and sentiment classification.
- Fraud and anomaly detection.

8. Advantages

- Provides high accuracy for classification and regression tasks.
- Performs well with high-dimensional datasets.
- Effective for both linear and non-linear data through kernel functions.
- Reduces overfitting by maximizing the decision margin.
- Suitable for small and medium-sized datasets.

9. Limitations

- Training can be computationally expensive for large datasets.
- Selection of kernel and hyperparameters significantly affects performance.
- Requires feature scaling for optimal results.
- Less efficient for extremely large datasets.
- Model interpretation is relatively difficult compared to simpler algorithms.

TCET
THAKUR COLLEGE OF
ENGINEERING & TECHNOLOGY

Code and Output:

Code:

```
#Imprting the Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import
train_test_split
from sklearn.preprocessing import LabelEncoder,
StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score,
confusion_matrix, classification_report

#Load Dataset
df = pd.read_csv("sales_data_sample.csv",
encoding='latin1')
df.head()
```

Output:

ORDERNUMBER	QUANTITYORDERED	PRICEEACH	SALES	STATUS	COUNTRY	DEALSIZE
10107	30	95.70	2871.00	Shipped	USA	Small
10121	34	81.35	2765.90	Shipped	France	Small
10134	41	94.74	3884.34	Shipped	France	Medium
10145	45	83.26	3746.70	Shipped	USA	Medium
10159	49	100.00	5205.27	Shipped	USA	Medium

Code:

```
#EDA

# Dataset Information
print("Shape:", df.shape)
print("\nMissing Values:\n", df.isnull().sum())
print("\nDuplicate Rows:", df.duplicated().sum())

# Statistical Summary
display(df.describe(include='all'))

# Visualizations
plt.figure(figsize=(15,4))

plt.subplot(1,3,1)
sns.countplot(x='STATUS', data=df)
plt.xticks(rotation=45)
plt.title("Order Status")

plt.subplot(1,3,2)
sns.histplot(df['SALES'], bins=25, kde=True)
plt.title("Sales Distribution")

plt.subplot(1,3,3)
sns.boxplot(x=df['SALES'])
plt.title("Sales Outliers")

plt.tight_layout()
plt.show()

# Correlation Matrix
plt.figure(figsize=(10,7))
sns.heatmap(df.corr(numeric_only=True),
cmap='coolwarm')
plt.title("Correlation Matrix")
plt.show()
```

Output:

Shape: (2823, 25)

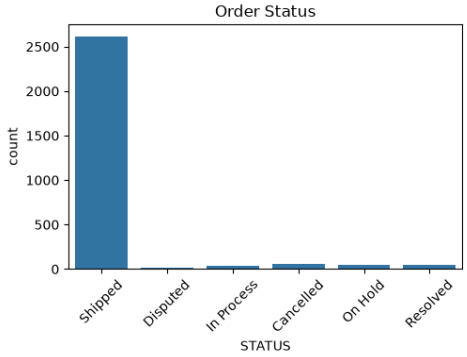
Missing Values:

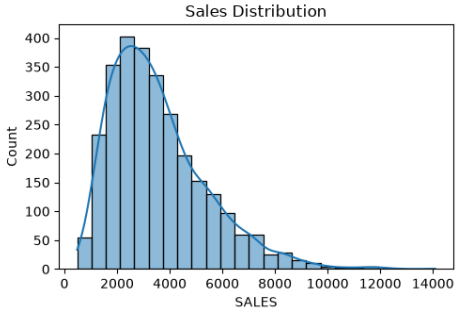
ORDERNUMBER	0
QUANTITYORDERED	0
PRICEEACH	0
ORDERLINENUMBER	0
SALES	0
ORDERDATE	0
STATUS	0
QTR_ID	0
MONTH_ID	0
YEAR_ID	0
PRODUCTLINE	0
MSRP	0
PRODUCTCODE	0
CUSTOMERNAME	0
PHONE	0
ADDRESSLINE1	0
ADDRESSLINE2	2521
CITY	0
STATE	1486
POSTALCODE	76
COUNTRY	0
TERRITORY	1074
...	
DEALSIZE	0

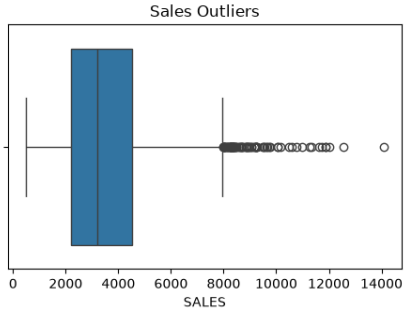
dtype: int64

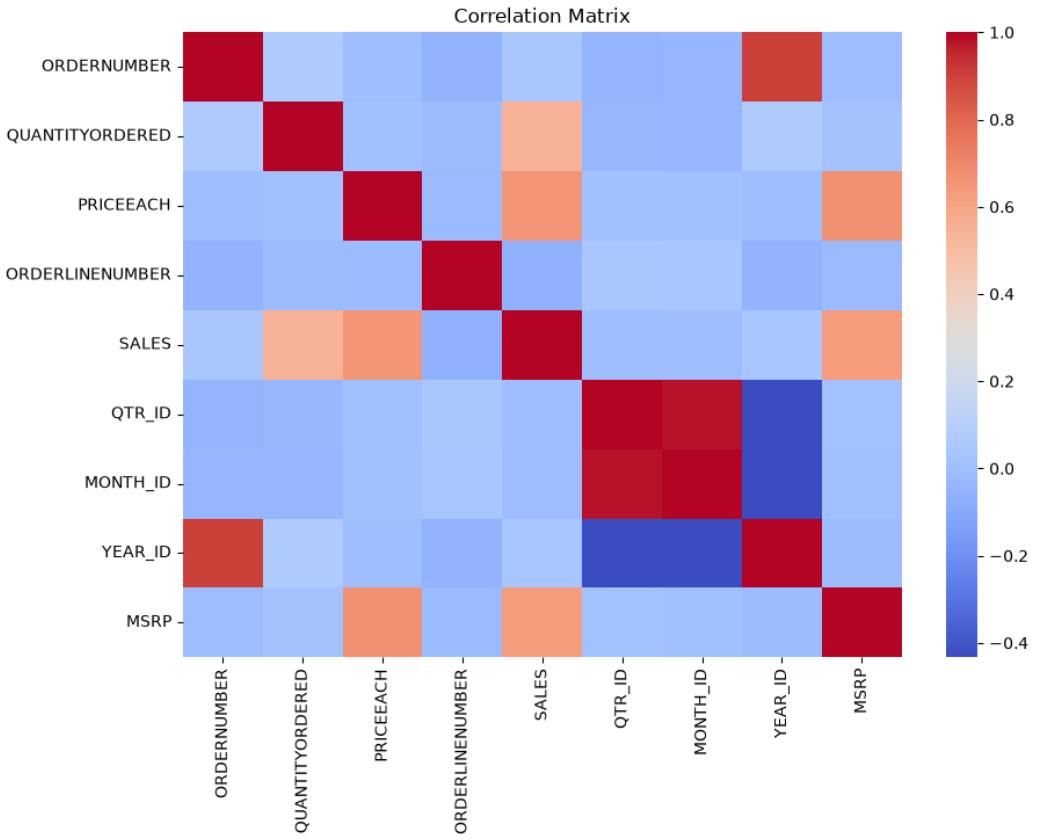
Duplicate Rows: 0

Statistic	ORDERNUMBER	QUANTITYORDERED	PRICEEACH	SALES	STATUS	DEALSIZE
Count	2823	2823	2823	2823	2823	2823
Unique	–	–	–	–	6	3
Top	–	–	–	–	Shipped	Medium
Frequency	–	–	–	–	2617	1384
Mean	10258.73	35.09	83.66	3553.89	–	–
Std. Dev.	92.09	9.74	20.17	1841.87	–	–
Minimum	10100	6	26.88	482.13	–	–
25%	10180	27	68.86	2203.43	–	–
50% (Median)	10262	35	95.70	3184.80	–	–
75%	10333.50	43	100.00	4508.00	–	–
Maximum	10425	97	100.00	14082.80	–	–









Code:

```
#Data Preprocessing
# Fill Missing Values
df['ADDRESSLINE2'].fillna("Unknown",
inplace=True)
df['STATE'].fillna("Unknown", inplace=True)
df['POSTALCODE'].fillna("Unknown",
inplace=True)
df['TERRITORY'].fillna("Unknown",
inplace=True)
# Remove Duplicates
df.drop_duplicates(inplace=True)
# Drop Unnecessary Columns
drop_cols = [
    'ORDERNUMBER','ORDERDATE','PHONE',
    'ADDRESSLINE1','ADDRESSLINE2',

'CUSTOMERNAME','CONTACTFIRSTNAME',
    'CONTACTLASTNAME','POSTALCODE',
    'PRODUCTCODE'
]
df.drop(columns=drop_cols, inplace=True)
# Encode Categorical Variables
encoder = LabelEncoder()
for col in
df.select_dtypes(include='object').columns:
    df[col] = encoder.fit_transform(df[col])
print("Preprocessing Completed")

#Feature Scaling
X = df.drop('STATUS', axis=1)
y = df['STATUS']
scaler = StandardScaler()
X = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

#Training SVM Model
svm = SVC(kernel='rbf', random_state=42)
svm.fit(X_train, y_train)
print("Model Trained Successfully")

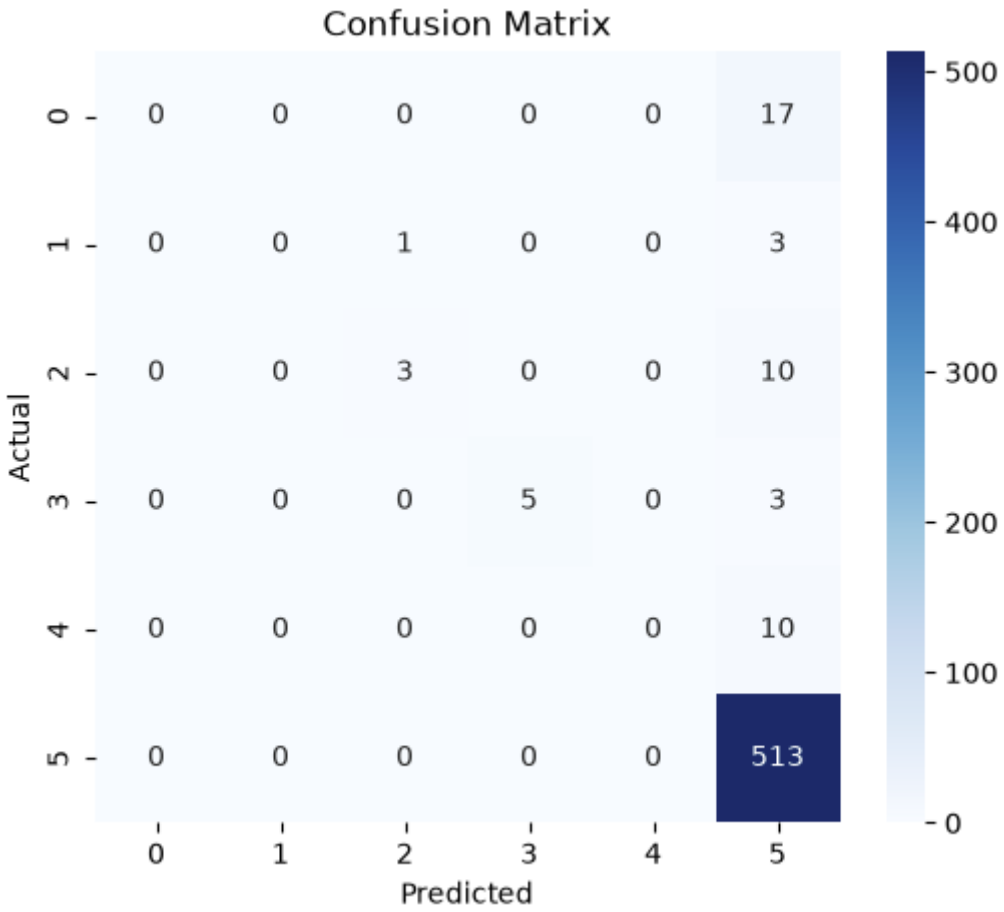
#Prediction
y_pred = svm.predict(X_test)
print("Accuracy:", accuracy_score(y_test,
y_pred))
print("\nClassification Report\n")
print(classification_report(y_test, y_pred))
```

Output:

Accuracy: 0.9221238938053097

Classification Report

	precision	recall	f1-score	support
0	0.00	0.00	0.00	17
1	0.00	0.00	0.00	4
2	0.75	0.23	0.35	13
3	1.00	0.62	0.77	8
4	0.00	0.00	0.00	10
5	0.92	1.00	0.96	513
accuracy			0.92	565
macro avg	0.45	0.31	0.35	565
weighted avg	**0.87**	**0.92**	**0.89**	**565**



Learning Outcomes: The student should have the ability to:
LO 4.1: Understand the working principle of Support Vector Machine (SVM) for solving classification, regression, and outlier detection problems.
LO 4.2: Implement Support Vector Machine models using appropriate kernel functions and perform feature preprocessing, including encoding and feature scaling.
LO 4.3: Train and test SVM models on real-world datasets to perform classification, regression, and anomaly detection tasks.
LO 4.4: Evaluate the performance of SVM models using appropriate evaluation metrics such as accuracy, confusion matrix, precision, recall, F1-score, and regression performance measures.

Course Outcomes: Design, implement, and evaluate Support Vector Machine (SVM) models for classification, regression, and outlier detection by applying appropriate preprocessing techniques, kernel functions, and evaluation metrics to solve real-world machine learning problems.

Conclusion:

THAKUR ZCT'S
TCET
THAKUR COLLEGE OF ENGINEERING & TECHNOLOGY

For Faculty Use: -

Correction Parameters	Formative Assessment [40%]	Timely completion of Practical [40%]	Attendance / Learning Attitude [20%]	
Marks Obtained				

Experiment 5

Aim: To implement AdaBoost Classifier and XGBoost algorithms for solving classification problems and evaluate their performance using appropriate machine learning evaluation metrics

Learning Objective: To understand the working principles of AdaBoost Classifier and XGBoost algorithms, implement these ensemble learning techniques for classification, and evaluate their performance using appropriate machine learning evaluation metrics.

Tools: VS Code, I-Python Notebook, Python, Pandas, NumPy, Scikit-learn, and XGBoost.

Theory: -

1. Introduction

Ensemble learning is a machine learning technique in which multiple models are combined to produce a stronger and more accurate predictive model. Boosting is an ensemble learning approach where weak learners are trained sequentially, and each new learner focuses on improving the errors made by the previous learners. AdaBoost and XGBoost are widely used boosting algorithms for solving classification and regression problems.

2. Boosting

Boosting combines several weak learners to create a strong learner. Unlike bagging, where models are trained independently, boosting trains models sequentially. Each successive model attempts to improve the performance of the previous model by giving greater attention to incorrectly predicted observations or by reducing the remaining prediction errors.

The general boosting process includes:

- Training an initial weak learner on the dataset.
- Identifying errors made by the model.
- Giving more attention to difficult or incorrectly predicted observations.
- Training the next learner to improve the previous model.
- Repeating the process for a specified number of iterations.
- Combining the predictions of all learners to produce the final result.

3. AdaBoost (Adaptive Boosting)

AdaBoost, or Adaptive Boosting, is an ensemble learning algorithm that combines multiple weak classifiers to form a strong classifier. It works by training models sequentially and adaptively changing the importance of training samples. Initially, all data points are assigned equal weights. After each iteration, the weights of incorrectly classified samples are increased so that the next weak learner focuses more on these difficult samples.

The main components of AdaBoost include:

- **Weak Learner:** A simple model, commonly a shallow decision tree or decision stump.
- **Sample Weights:** Values assigned to training samples to indicate their importance.
- **Classifier Weight:** The importance assigned to each weak learner based on its performance.

- **Strong Classifier:** The final model obtained by combining the predictions of all weak learners.

4. Working of AdaBoost

The AdaBoost algorithm works in the following steps:

1. Equal weights are assigned to all training samples.
2. A weak classifier is trained on the weighted dataset.
3. The predictions of the classifier are compared with the actual class labels.
4. The error of the weak classifier is calculated.
5. Higher weights are assigned to incorrectly classified samples, while correctly classified samples receive relatively less emphasis.
6. A new weak classifier is trained with greater focus on the difficult samples.
7. The process is repeated for a predefined number of estimators.
8. The predictions of all weak classifiers are combined using weighted voting to produce the final classification result.

5. Advantages of AdaBoost

- Improves the performance of weak classifiers by combining them into a strong model.
- Focuses on difficult and misclassified samples during training.
- Can provide high classification accuracy.
- Simple and effective for many supervised learning problems.
- Can be implemented using different weak learners.

6. Limitations of AdaBoost

- Sensitive to noisy data and outliers.
- Incorrectly classified noisy observations may receive increased importance.
- Performance depends on the selection of appropriate weak learners.
- Training may become computationally expensive when a large number of estimators is used.

7. XGBoost (Extreme Gradient Boosting)

XGBoost stands for Extreme Gradient Boosting. It is an optimized implementation of the gradient boosting technique designed for efficiency, speed, and high predictive performance. XGBoost uses decision trees as base learners and builds them sequentially, with each new tree attempting to correct the errors made by the previous trees. It also includes regularization to control model complexity and improve generalization.

8. Working of XGBoost

The XGBoost algorithm works by sequentially building decision trees to reduce the errors of previous predictions. The general process includes:

1. An initial prediction is generated using a base model.
2. The difference between actual and predicted values is calculated using a loss function.
3. A new decision tree is trained to reduce the previous errors.
4. The predictions of the new tree are added to the predictions from earlier trees.
5. Regularization is applied to control the complexity of the trees and reduce overfitting.

6. The process continues until a specified number of boosting rounds or another stopping criterion is reached.
7. The final prediction is obtained by combining the contributions of all decision trees.

Mathematically, the XGBoost prediction can be represented as:

$$\hat{y}_i = \Sigma f_k(x_i)$$

where:

- $\hat{y}_i$ represents the final predicted value.
- $f_k(x_i)$ represents the prediction of the k-th decision tree.
- **K** represents the total number of trees in the ensemble.

XGBoost optimizes an objective function consisting of a loss function and a regularization term:

Objective = Loss Function + Regularization Term

The loss function measures the difference between actual and predicted values, while the regularization term penalizes overly complex models.

9. Important Features of XGBoost

- **Regularization:** Helps control model complexity and reduce overfitting.
- **Sequential Learning:** Each tree is trained to improve the errors of previous trees.
- **Parallel Processing:** Optimizations allow efficient processing during model training.
- **Missing Value Handling:** Supports handling missing values during tree construction and prediction.
- **Feature Importance:** Can identify the relative importance of input features.
- **Scalability:** Suitable for handling large and complex datasets.

10. Advantages of XGBoost

- Provides high predictive performance for many machine learning tasks.
- Efficient and scalable for large datasets.
- Uses regularization to improve generalization and reduce overfitting.
- Supports parallel processing and efficient computation.
- Can handle missing values.
- Provides feature importance for analyzing model behavior.
- Can be used for both classification and regression problems.

11. Limitations of XGBoost

- Can be computationally intensive on systems with limited resources.
- Requires proper hyperparameter tuning for optimal performance.
- May overfit when too many trees or inappropriate parameter values are used.
- Can be sensitive to noisy data and outliers.
- Is generally more difficult to interpret than simpler machine learning models.

12. AdaBoost and XGBoost Comparison

Feature	AdaBoost	XGBoost
Full Form	Adaptive Boosting	Extreme Gradient Boosting
Main Approach	Increases focus on misclassified samples	Sequentially minimizes prediction errors using gradient boosting
Base Learners	Commonly shallow decision trees	Decision trees
Error Handling	Adjusts sample weights	Optimizes a loss function
Regularization	Limited compared to XGBoost	Includes regularization to control complexity
Speed and Scalability	Suitable for smaller and moderate problems	Optimized for speed and large datasets
Overfitting Control	Depends on weak learners and parameters	Uses regularization and other optimization techniques
Applications	Classification and regression	Classification and regression

The comparison shows that both algorithms use the boosting approach and sequentially improve model performance. AdaBoost mainly emphasizes incorrectly classified samples through adaptive weighting, whereas XGBoost uses gradient-based optimization and regularization to achieve efficient and accurate predictions.

13. Applications

AdaBoost and XGBoost can be applied in various real-world machine learning problems, including:

- Fraud detection.
- Customer churn prediction.
- Medical diagnosis.
- Spam and email classification.
- Credit risk analysis.
- Sales and demand prediction.
- Financial forecasting.
- Image and pattern classification.
- Recommendation and ranking systems.
- General classification and regression tasks.

Code and Output:

Code:

```
# Importing Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
from xgboost import XGBClassifier
```

Output: Libraries Imported Successfully

Code:

```
# Load Dataset
df = pd.read_csv("sales_data_sample.csv", encoding="latin1")
print("Dataset Loaded Successfully")
print(df.head())
```

Output:

	ORDERNUMBER	QUANTITYORDERED	PRICEEACH	ORDERLINENUMBER	SALES	ORDERDATE	STATUS	QTR_ID	MONTH_ID	YEAR_ID	...	ADDRESSLINE1
0	10107	30	95.70	2	2871.00	2/24/2003 0:00	Shipped	1	2	2003	...	897 Long Airport Avenue
1	10121	34	81.35	5	2765.90	5/7/2003 0:00	Shipped	2	5	2003	...	59 rue de l'Abbaye
2	10134	41	94.74	2	3884.34	7/1/2003 0:00	Shipped	3	7	2003	...	27 rue du Colonel Pierre Avia
3	10145	45	83.26	6	3746.70	8/25/2003 0:00	Shipped	3	8	2003	...	78934 Hillside Dr.
4	10159	49	100.00	14	5205.27	10/10/2003 0:00	Shipped	4	10	2003	...	7734 Strong St.

ADDRESSLINE2	CITY	STATE	POSTALCODE	COUNTRY	TERRITORY	CONTACTLASTNAME	CONTACTFIRSTNAME	DEALSIZE
NaN	NYC	NY	10022	USA	NaN	Yu	Kwai	Small
NaN	Reims	NaN	51100	France	EMEA	Henriot	Paul	Small
NaN	Paris	NaN	75508	France	EMEA	Da Cunha	Daniel	Medium
NaN	Pasadena	CA	90003	USA	NaN	Young	Julie	Medium
NaN	San Francisco	CA	NaN	USA	NaN	Brown	Julie	Medium

THAKUR COLLEGE OF ENGINEERING & TECHNOLOGY

Code:

```
# Dataset Information
print("\nShape:", df.shape)
print("\nMissing Values:\n", df.isnull().sum())
print("\nDuplicate Rows:", df.duplicated().sum())
```

Output:

Shape: (2823, 25)

Missing Values:

ORDERNUMBER	0
QUANTITYORDERED	0
PRICEEACH	0
ORDERLINENUMBER	0
SALES	0
ORDERDATE	0
STATUS	0
QTR_ID	0
MONTH_ID	0
YEAR_ID	0
PRODUCTLINE	0

```
MSRP                0
PRODUCTCODE         0
CUSTOMERNAME        0
PHONE               0
ADDRESSLINE1        0
ADDRESSLINE2     2521
CITY                0
STATE            1486
POSTALCODE         76
COUNTRY             0
```

Code:

```
# Data Preprocessing

# Fill Missing Values
df['ADDRESSLINE2'] =
df['ADDRESSLINE2'].fillna("Unknown")
df['STATE'] = df['STATE'].fillna("Unknown")
df['POSTALCODE'] =
df['POSTALCODE'].fillna("Unknown")
df['TERRITORY'] =
df['TERRITORY'].fillna("Unknown")


# Remove Duplicate Rows
df.drop_duplicates(inplace=True)


# Drop Unnecessary Columns
drop_cols = [
    'ORDERNUMBER',
    'ORDERDATE',
    'PHONE',
    'ADDRESSLINE1',
    'ADDRESSLINE2',
    'CUSTOMERNAME',
    'CONTACTFIRSTNAME',
    'CONTACTLASTNAME',
    'POSTALCODE',
    'PRODUCTCODE'
]

df.drop(columns=drop_cols, inplace=True)


# Encode Categorical Variables
encoder = LabelEncoder()

for col in
df.select_dtypes(include='object').columns:
    df[col] = encoder.fit_transform(df[col])


print("\nPreprocessing Completed")
```

Output:

```
Preprocessing Completed
```

Code:

```
# Separate Features and Target
X = df.drop('STATUS', axis=1)
y = df['STATUS']


# Encode Target Variable
target_encoder = LabelEncoder()
y = target_encoder.fit_transform(y)


# Train-Test Split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("\nTraining Data:", X_train.shape)
print("Testing Data:", X_test.shape)
```

Output:

```
Training Data: (2258, 14)
Testing Data: (565, 14)
```

Code:

```
# ADABOOST CLASSIFIER

# Create Weak Learner
base_model = DecisionTreeClassifier(
    max_depth=1,
    random_state=42
)

# Create AdaBoost Model
ada_model = AdaBoostClassifier(
    estimator=base_model,
    n_estimators=100,
    learning_rate=1.0,
    random_state=42
)

# Train AdaBoost
ada_model.fit(X_train, y_train)

# Prediction
ada_pred = ada_model.predict(X_test)

# Evaluation
ada_accuracy = accuracy_score(y_test, ada_pred)

print("\n===== AdaBoost Classifier =====")
print("Accuracy:", ada_accuracy)

print("\nClassification Report:")
print(classification_report(
    y_test,
    ada_pred,
    zero_division=0
))


# Confusion Matrix
plt.figure(figsize=(7, 5))
sns.heatmap(
    confusion_matrix(y_test, ada_pred),
    annot=True,
    fmt='d',
    cmap='Blues'
)
plt.title("AdaBoost Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

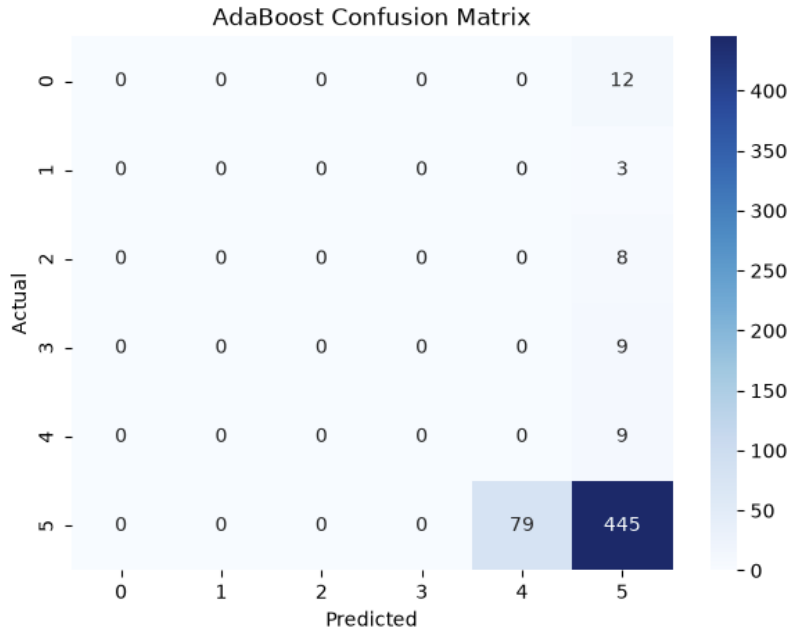
Output:

```
===== AdaBoost Classifier =====
Accuracy: 0.7876106194690266

Classification Report:
              precision    recall  f1-score   support

           0       0.00      0.00      0.00        12
           1       0.00      0.00      0.00         3
           2       0.00      0.00      0.00         8
           3       0.00      0.00      0.00         9
           4       0.00      0.00      0.00         9
           5       0.92      0.85      0.88       524

    accuracy                           0.79       565
   macro avg       0.15      0.14      0.15       565
weighted avg       0.85      0.79      0.82       565
```



Code:

```
# XGBOOST CLASSIFIER

# Create XGBoost Model
xgb_model = XGBClassifier(
    n_estimators=100,
    max_depth=4,
    learning_rate=0.1,
    subsample=0.8,
    colsample_bytree=0.8,
    objective='multi:softmax',
    num_class=len(np.unique(y)),
    eval_metric='mlogloss',
    random_state=42
)

# Train XGBoost
xgb_model.fit(X_train, y_train)

# Prediction
xgb_pred = xgb_model.predict(X_test)

# Evaluation
xgb_accuracy = accuracy_score(y_test, xgb_pred)

print("\n===== XGBoost Classifier =====")
print("Accuracy:", xgb_accuracy)

print("\nClassification Report:")
print(classification_report(
    y_test,
    xgb_pred,
    zero_division=0
))

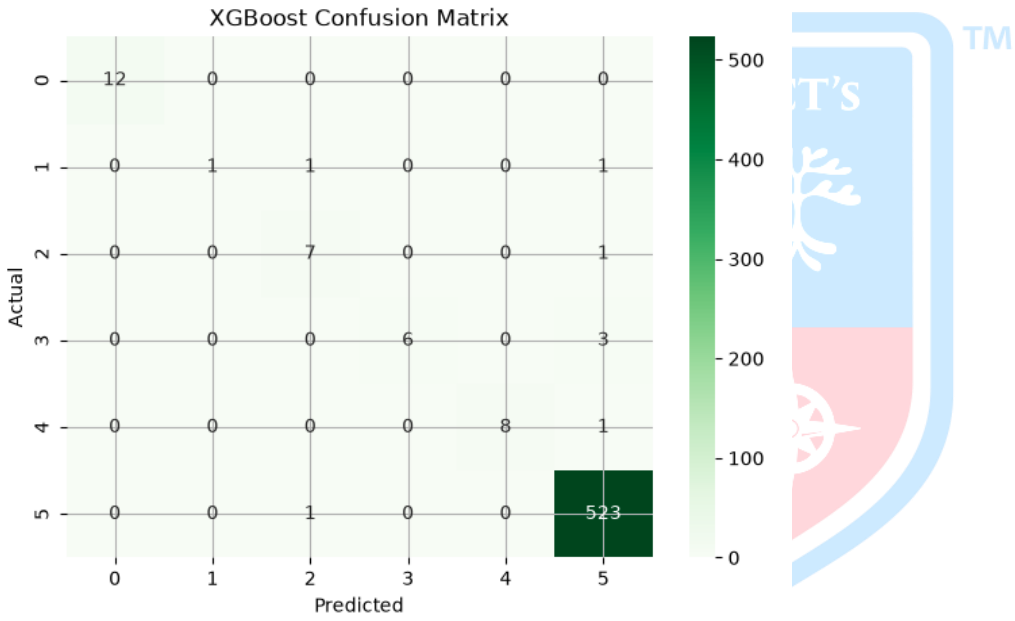
# Confusion Matrix
plt.figure(figsize=(7, 5))
sns.heatmap(
    confusion_matrix(y_test, xgb_pred),
    annot=True,
    fmt='d',
    cmap='Greens'
)
plt.title("XGBoost Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

Output:

```
===== XGBoost Classifier =====
Accuracy: 0.9858407079646018

Classification Report:
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	12
1	1.00	0.33	0.50	3
2	0.78	0.88	0.82	8
3	1.00	0.67	0.80	9
4	1.00	0.89	0.94	9
5	0.99	1.00	0.99	524
accuracy			0.99	565
macro avg	0.96	0.79	0.84	565
weighted avg	0.99	0.99	0.98	565



Code:

```
# MODEL COMPARISON
print("\n===== Model Comparison =====")
print("AdaBoost Accuracy :", ada_accuracy)
print("XGBoost Accuracy  :", xgb_accuracy)
if ada_accuracy > xgb_accuracy:
    print("\nAdaBoost performed better on the test dataset.")
elif xgb_accuracy > ada_accuracy:
    print("\nXGBoost performed better on the test dataset.")
else:
    print("\nBoth models achieved the same accuracy.")
```

Output:

```
 ===== Model Comparison =====
AdaBoost Accuracy : 0.7876106194690266
XGBoost Accuracy  : 0.9858407079646018
XGBoost performed better on the test dataset.
```

Learning Outcomes: The student should have the ability to:
LO 5.1: Understand the working principles of AdaBoost and XGBoost algorithms for solving classification problems.
LO 5.2: Implement AdaBoost and XGBoost models using appropriate data preprocessing and feature selection techniques.
LO 5.3: Train and test boosting-based machine learning models on real-world datasets and generate predictions.
LO 5.4: Evaluate and compare the performance of AdaBoost and XGBoost models using appropriate evaluation metrics such as accuracy, confusion matrix, precision, recall, and F1-score.

Course Outcomes: Design, implement, and evaluate AdaBoost and XGBoost ensemble learning models by applying appropriate data preprocessing techniques and performance evaluation metrics to solve real-world machine learning classification problems.

Conclusion:

THAKUR ZCT'S
TCET
THAKUR COLLEGE OF ENGINEERING & TECHNOLOGY

For Faculty Use: -

Correction Parameters	Formative Assessment [40%]	Timely completion of Practical [40%]	Attendance / Learning Attitude [20%]	
Marks Obtained				

Experiment 6

Aim: Implementation of K-Means Clustering

Learning Objective: To understand the concept of unsupervised learning and implement the K-Means Clustering algorithm to group similar data points into different clusters based on their features, and to analyze and interpret the clusters formed using appropriate visualization techniques.

Tools: VS Code, I-Python Notebook.

Theory: -

1. Introduction

Machine learning algorithms are broadly divided into supervised and unsupervised learning. In supervised learning, every training example has a known output label, and the model learns to map inputs to those labels. In many real-world situations, however, data is available without any labels — for example, a list of customers, players or documents with no predefined categories. Unsupervised learning deals with such unlabelled data and tries to discover hidden structure, patterns or groupings within it.

Clustering is the most common unsupervised learning task. Its goal is to divide a dataset into groups, called clusters, such that data points within the same cluster are highly similar to each other while points in different clusters are dissimilar. K-Means Clustering is one of the simplest and most widely used clustering algorithms. It is popular because it is easy to understand, computationally efficient and works well on large datasets with numerical features.

2. K-Means Clustering

K-Means is a centroid-based clustering algorithm. It divides the dataset into a fixed number of clusters, denoted by K, which must be decided before running the algorithm. Each cluster is represented by a single central point known as the centroid, which is calculated as the mean (average) of all the data points belonging to that cluster. This is why the algorithm is called K-"Means".

Every data point is assigned to exactly one cluster — the cluster whose centroid is nearest to it. Because the assignment depends on distance, K-Means naturally forms compact, roughly spherical groups around each centroid. The algorithm repeatedly improves the position of the centroids until the clusters no longer change, at which point the model is said to have converged.

The main components of K-Means include:

- **K (Number of Clusters):** A user-defined value that specifies how many groups the data should be divided into.
- **Centroid:** The centre of a cluster, computed as the mean of all data points assigned to it.
- **Distance Measure:** Usually Euclidean distance, used to determine which centroid a data point is closest to.
- **Cluster Label:** The cluster number (0, 1, 2, …) assigned to each data point after clustering.
- **Inertia (WCSS):** The Within-Cluster Sum of Squares, which measures how tightly the points are grouped around their centroids.

3. Working of K-Means

K-Means follows an iterative refinement procedure that alternates between two main steps: an assignment step and an update step. Initially, K centroids are placed in the feature space. In scikit-learn, this is done using the k-means++ strategy, which spreads the initial centroids far apart to obtain better and more stable results than purely random initialisation.

In the assignment step, the distance between each data point and every centroid is calculated, and the point is assigned to the nearest centroid. In the update step, each centroid is moved to the mean position of all the points that were assigned to it. These two steps are repeated until the centroids stop moving significantly or a maximum number of iterations is reached. Since the final result can depend on the starting centroids, the algorithm is usually run several times with different initialisations (controlled by the n_init parameter), and the run with the lowest inertia is selected.

The clustering process includes:

- Choosing the number of clusters K.
- Initialising K centroids.
- Assigning each data point to its nearest centroid.
- Recalculating each centroid as the mean of its assigned points.
- Repeating the assignment and update steps until convergence.
- Obtaining the final cluster labels and centroids.

4. Objective Function

The goal of K-Means is to make each cluster as compact as possible. Mathematically, it minimises the Within-Cluster Sum of Squares (WCSS), which is the total squared distance between every data point and the centroid of the cluster it belongs to:

$$J = \Sigma\ (k = 1 \text{ to } K)\ \Sigma\ (x \in C_k)\ \|x - \mu_k\|^2$$

Here, C_k is the set of data points in cluster k, x is a data point, and μ_k is the centroid of cluster k. Each assignment step and each update step can only decrease (or keep constant) the value of J, which guarantees that the algorithm converges. A smaller value of J indicates tighter and more well-defined clusters. However, K-Means finds a local minimum, not necessarily the global minimum, which is why multiple initialisations are used.

5. Choosing the Value of K

Since K must be specified before training, selecting a suitable value is one of the most important decisions in K-Means. If K is too small, genuinely different groups are merged together; if K is too large, natural groups are split into meaningless fragments. Several methods are used to choose an appropriate value:

- **Elbow Method:** WCSS is plotted for different values of K. WCSS always decreases as K increases, but the point where the rate of decrease sharply slows down (the "elbow") is chosen as the optimal K.
- **Silhouette Score:** Measures how similar a point is to its own cluster compared to the nearest other cluster. The score ranges from −1 to 1, and higher values indicate better-separated clusters.
- **Domain Knowledge:** The number of meaningful groups may already be known from the problem. In this experiment, K = 3 is chosen to represent three performance tiers of chess candidates.

6. Importance of Feature Scaling

K-Means relies entirely on distance calculations, so the scale of each feature has a direct effect on the clusters formed. Features with large numerical ranges dominate the distance measure and effectively decide the clusters on their own. In this experiment, the Elo rating ranges from about 2400 to 2880, while the win ratio lies between 0 and 1. Without scaling, the win ratio would have almost no influence on the clustering.

To avoid this, the features are standardized using StandardScaler, which transforms each feature to have a mean of zero and a standard deviation of one using $Z = (X - \mu) / \sigma$. After standardization, both Elo rating and win ratio contribute equally to the distance, and the resulting clusters reflect both playing strength and performance. The cluster centroids reported by the model are therefore in scaled coordinates.

7. Applications of K-Means Clustering

Because it is simple and scalable, K-Means is used in a wide variety of fields to discover natural groupings in data. Some common applications include:

- Customer segmentation for targeted marketing.
- Grouping players or teams by performance level in sports analytics.
- Image segmentation and colour quantization.
- Document and news article clustering.
- Anomaly detection by identifying points far from any centroid.
- Recommendation systems and market basket grouping.

8. Advantages

- Simple to understand and easy to implement.
- Computationally efficient and scales well to large datasets.
- Produces compact clusters that are easy to interpret through their centroids.
- Guaranteed to converge in a finite number of iterations.
- Works very well when clusters are roughly spherical and well separated.

9. Limitations

- The number of clusters K must be specified in advance.
- Results depend on the initial placement of centroids and may reach a local minimum.
- Sensitive to outliers, which can pull centroids away from the true cluster centre.
- Requires feature scaling, since it is based on distance.
- Assumes spherical clusters of similar size and cannot handle complex, non-convex shapes.

Code and Output:

Code:

```
#Importing the Libraries
import pandas as pd
import numpy as np
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

# Step 1: Generate 1000 entries of Chess Candidates Dataset
np.random.seed(42)
n_samples = 1000
df = pd.DataFrame({
    'Elo_Rating': np.random.randint(2400, 2880, n_samples),
    'Rapid_Rating': np.random.randint(2300, 2850, n_samples),
    'Blitz_Rating': np.random.randint(2300, 2900, n_samples),
    'Wins': np.random.randint(0, 15, n_samples),
    'Draws': np.random.randint(2, 20, n_samples),
    'Losses': np.random.randint(0, 10, n_samples),
})
df['Total_Games'] = df['Wins'] + df['Draws'] + df['Losses']
df['Win_Ratio'] = df['Wins'] / df['Total_Games']
df.head()
```

Output:

	Elo_Rating	Rapid_Rating	Blitz_Rating	Wins	Draws	Losses	Total_Games	Win_Ratio
0	2502	2300	2680	0	16	5	21	0.0000
1	2835	2350	2401	8	9	2	19	0.4211
2	2748	2745	2793	13	5	7	25	0.5200
3	2670	2748	2318	11	3	5	19	0.5789
4	2506	2787	2753	2	11	0	13	0.1538

Code:

```
# Step 2: Select features for clustering (Elo Rating vs Win Ratio)
X_cluster = df[['Elo_Rating', 'Win_Ratio']]

# Step 3: Standardize the features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_cluster)

# Step 4: Fit K-Means Model (K=3 for candidate performance tiers)
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
labels = kmeans.fit_predict(X_scaled)
centroids = kmeans.cluster_centers_

print("First 50 Cluster Labels:")
print(labels[:50])
print("\nCluster Centroids (Scaled Coordinates):")
print(centroids)
```

Output:

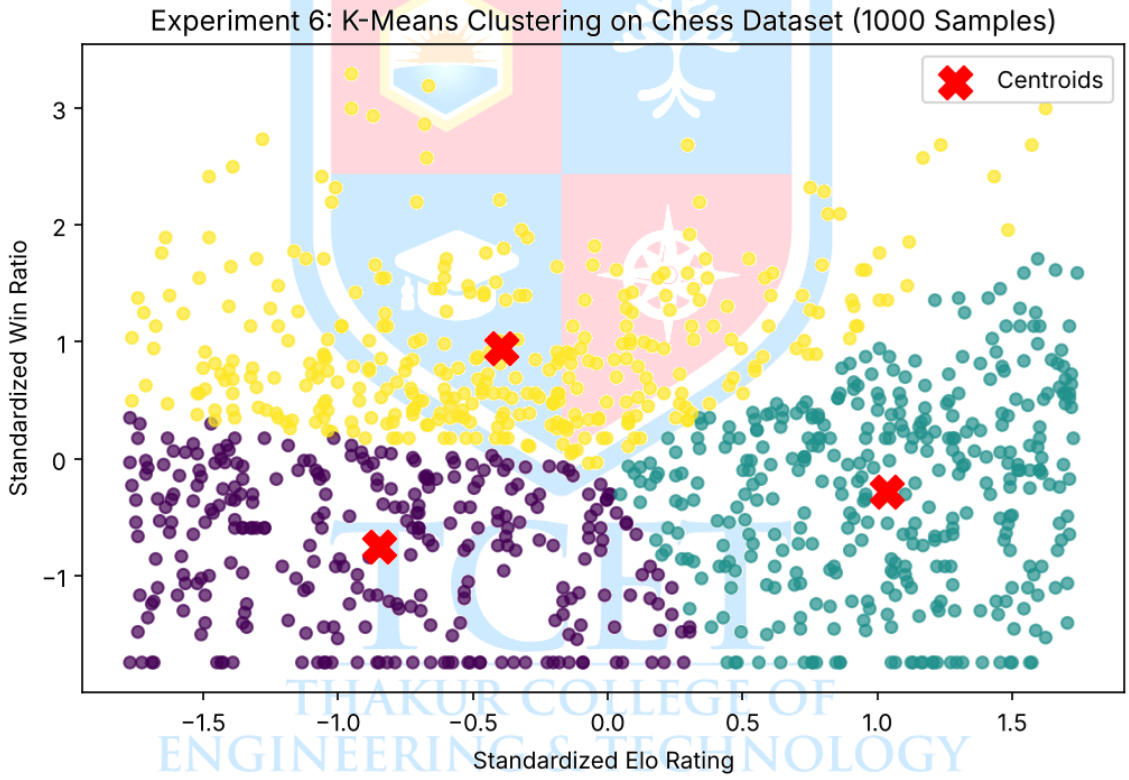
```
First 50 Cluster Labels:
[0 1 2 2 0 2 0 2 2 2 1 0 1 1 0 2 2 1 0 2 0 1 2 1 1 2 1 0 1 2 2 1 1 0 2 2 2
```

```
 0 1 0 0 1 2 2 2 2 1 0 1 2]
Cluster Centroids (Scaled Coordinates):
[[-0.84595017 -0.75585696]
 [ 1.03586742 -0.28003095]
 [-0.39597569  0.94723099]]
```

Code:

```
# Step 5: Visualize Clusters
plt.figure(figsize=(8, 5))
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=labels,
s=25, cmap='viridis', alpha=0.7)
plt.scatter(centroids[:, 0], centroids[:, 1],
marker='X', s=200, c='red', label='Centroids')
plt.xlabel("Standardized Elo Rating")
plt.ylabel("Standardized Win Ratio")
plt.title("Experiment 6: K-Means Clustering on
Chess Dataset (1000 Samples)")
plt.legend()
plt.show()
```

Output:



Learning Outcomes: The student should have the ability to:
LO 6.1: Understand the concept of unsupervised learning and the working principle of the K-Means Clustering algorithm.
LO 6.2: Implement K-Means Clustering after performing feature selection and feature scaling using standardization.
LO 6.3: Determine an appropriate number of clusters and interpret the cluster centroids obtained from the given dataset.
LO 6.4: Analyze and visualize the clusters formed to draw meaningful insights about groups of similar data points.

Course Outcomes: Design, implement, and analyze K-Means Clustering models by applying appropriate preprocessing techniques such as feature scaling, selecting a suitable number of clusters, and interpreting cluster centroids and visualizations to discover meaningful patterns in unlabelled real-world data.

Conclusion:

TCET
THAKUR COLLEGE OF ENGINEERING & TECHNOLOGY

For Faculty Use: -

Correction Parameters	Formative Assessment [40%]	Timely completion of Practical [40%]	Attendance / Learning Attitude [20%]	
Marks Obtained				

Experiment 7

Aim: Implementation of PCA for Dimensionality Reduction

Learning Objective: To understand the principles of dimensionality reduction using Principal Component Analysis (PCA), project high-dimensional data into a lower-dimensional space, and evaluate the performance of a machine learning classifier (KNN) on the dimensionally reduced feature set along with visualizing the data along the top principal components.

Tools: VS Code, I-Python Notebook.

Theory: -

1. Introduction

Real-world datasets often contain a large number of features. Many of these features are correlated with each other and carry overlapping information — for example, a chess player's classical, rapid and blitz ratings all reflect overall playing strength. As the number of features grows, the data becomes sparse, computation becomes slower, visualization becomes impossible beyond three dimensions, and machine learning models become more prone to overfitting. This problem is commonly known as the curse of dimensionality.

Dimensionality reduction techniques address this problem by representing the data with fewer variables while preserving as much useful information as possible. There are two main approaches: feature selection, which keeps a subset of the original features, and feature extraction, which creates new features from combinations of the original ones. Principal Component Analysis (PCA) is the most widely used feature extraction technique.

2. Principal Component Analysis (PCA)

Principal Component Analysis (PCA) is an unsupervised dimensionality reduction technique that projects high-dimensional data into a lower-dimensional subspace while retaining as much variance as possible. It is called unsupervised because it uses only the input features and does not consider the target labels. PCA transforms a set of possibly correlated variables into a new set of linearly uncorrelated variables called principal components.

Each principal component is a linear combination of the original features. The first principal component (PC1) points in the direction along which the data varies the most. The second principal component (PC2) captures the next largest amount of variance while being orthogonal (perpendicular) to PC1, and so on. By keeping only the first few components, the data can be represented in far fewer dimensions with minimal loss of information.

The main components of PCA include:

- **Principal Components:** New orthogonal axes formed as linear combinations of the original features.
- **Eigenvectors:** Define the direction of each principal component in the original feature space.
- **Eigenvalues:** Indicate the amount of variance captured along each principal component.
- **Explained Variance Ratio:** The fraction of the total variance retained by each component.
- **Loadings:** The weights showing how strongly each original feature contributes to a component.

3. Working of PCA

PCA is carried out through a sequence of mathematical steps that convert the original data into a new coordinate system defined by the principal components.

Step 1 – Standardization: Each feature is scaled so that it has a mean of zero and a variance of one. This ensures that features measured on large scales do not dominate the analysis:

$$Z = (X - \mu) / \sigma$$

Step 2 – Covariance Matrix Computation: The covariance matrix is calculated to capture how every pair of features varies together. Positive values indicate features that increase together, and negative values indicate features that move in opposite directions:

$$\Sigma = (1 / (n - 1)) \cdot Z^T Z$$

Step 3 – Eigenvalue and Eigenvector Decomposition: The eigenvalues (λ) and eigenvectors (v) of the covariance matrix are computed. Each eigenvector gives the direction of a principal component, and its eigenvalue gives the variance along that direction:

$$\Sigma v = \lambda v$$

Step 4 – Selecting Principal Components: The eigenvalues are sorted in descending order, and the top k eigenvectors corresponding to the largest eigenvalues are selected. These eigenvectors form the columns of the projection matrix W.

Step 5 – Projecting the Data: The standardized data matrix Z is multiplied by the projection matrix W to obtain the data in the new k-dimensional feature space:

$$X_pca = Z \cdot W$$

4. Explained Variance

The explained variance ratio of a component is its eigenvalue divided by the sum of all eigenvalues. It shows what percentage of the information in the original data is retained by that component. The cumulative explained variance of the first k components indicates how much information is preserved after reducing the data to k dimensions.

When the original features are strongly correlated, a small number of components can capture most of the variance. When the features are largely independent of each other — as in a randomly generated dataset — the variance is spread more evenly across components, and two components retain a smaller share of the total information. Examining the explained variance therefore helps judge how much is lost during reduction.

5. PCA with K-Nearest Neighbors (KNN)

PCA is commonly used as a preprocessing step before training a classifier. In this experiment, a K-Nearest Neighbors (KNN) classifier is trained on the two principal components instead of the original features. KNN classifies a new data point by finding the K closest training points and assigning the class that appears most

frequently among them. Since KNN is distance-based, it benefits from standardized inputs and from reducing the number of dimensions.

The accuracy of the classifier on the reduced data shows how well the class structure is preserved after projection. If the principal components retain the features that separate the classes, accuracy remains high; if important information is discarded, accuracy drops. The scatter plot of PC1 against PC2, coloured by class, provides a visual check of how well the classes are separated in the reduced space.

The process includes:

- Generating and preparing the dataset.
- Creating the target class labels (player tiers).
- Standardizing all numerical features.
- Applying PCA to obtain two principal components.
- Splitting the reduced data into training and testing sets.
- Training the KNN classifier and evaluating its accuracy.

6. Choosing the Number of Components

Selecting the number of components is a trade-off between simplicity and information retained. Too few components may discard useful information, while too many reduce the benefit of dimensionality reduction. Common approaches include:

- **Cumulative Explained Variance:** Select the smallest number of components that retain a desired percentage (for example 90–95%) of the total variance.
- **Scree Plot:** Plot the eigenvalues against the component number and choose the point where the curve flattens.
- **Visualization Needs:** Two or three components are chosen when the main goal is to plot and visually inspect the data.

7. Applications of PCA

PCA is used across many domains wherever high-dimensional data must be simplified, visualized or compressed. Some common applications include:

- Visualization of high-dimensional datasets in 2D or 3D.
- Image compression and face recognition (Eigenfaces).
- Noise reduction in signals and images.
- Feature extraction before training machine learning models.
- Genomics and bioinformatics data analysis.
- Finance and sports performance analytics.

8. Advantages

- Reduces the number of features, lowering computation time and memory usage.
- Removes multicollinearity by producing uncorrelated components.
- Makes it possible to visualize high-dimensional data in 2D or 3D.
- Reduces noise and can help prevent overfitting.
- Unsupervised, so it does not require class labels.

9. Limitations

- Principal components are difficult to interpret in terms of the original features.
- Some information is lost when too few components are retained.
- Captures only linear relationships between features.
- Sensitive to feature scaling and to outliers.
- Maximizing variance does not always maximize class separability.

TCET
THAKUR COLLEGE OF
ENGINEERING & TECHNOLOGY

Code and Output:

Code:

```
#Importing the Libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import
train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.neighbors import
KNeighborsClassifier
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

# Step 1: Generate 1000 entries of Chess Dataset
np.random.seed(42)
n_samples = 1000
df = pd.DataFrame({
    'Elo_Rating': np.random.randint(2400, 2880,
n_samples),
    'Rapid_Rating': np.random.randint(2300, 2850,
n_samples),
    'Blitz_Rating': np.random.randint(2300, 2900,
n_samples),
    'Age': np.random.randint(16, 48, n_samples),
    'Wins': np.random.randint(0, 15, n_samples),
    'Draws': np.random.randint(2, 20, n_samples),
    'Losses': np.random.randint(0, 10, n_samples),
})
df['Total_Games'] = df['Wins'] + df['Draws'] +
df['Losses']
df['Win_Ratio'] = df['Wins'] / df['Total_Games']
df.head()
```

Output:

	Elo_Rating	Rapid_Rating	Blitz_Rating	Age	Wins	Draws	Losses	Total_Games	Win_Ratio
0	2502	2300	2680	32	1	9	7	17	0.0588
1	2835	2350	2401	24	7	12	5	24	0.2917
2	2748	2745	2793	45	12	2	2	16	0.7500
3	2670	2748	2318	27	8	7	4	19	0.4211
4	2506	2787	2753	18	12	17	1	30	0.4000

THAKUR COLLEGE OF ENGINEERING & TECHNOLOGY

Code:

```
# Step 2: Create Target Label for Classification (3 Player Tiers)
def assign_tier(row):
    if row['Elo_Rating'] >= 2760 and row['Win_Ratio'] >= 0.35:
        return 2
    elif row['Elo_Rating'] >= 2650 and row['Win_Ratio'] >= 0.20:
        return 1
    else:
        return 0

y = df.apply(assign_tier, axis=1)
X = df.copy()

# Step 3: Standardize the full feature set
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Step 4: Apply PCA (Reduce 9 features down to 2 Principal Components)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# Step 5: Split Train/Test sets on reduced features
X_train_pca, X_test_pca, y_train_pca, y_test_pca = train_test_split(
    X_pca, y, test_size=0.2, random_state=42, stratify=y
)

# Step 6: Train k-NN Classifier on reduced components
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_pca, y_train_pca)
y_pred_pca = knn.predict(X_test_pca)

accuracy_pca = accuracy_score(y_test_pca, y_pred_pca)
print(f"Accuracy of k-NN on PCA-reduced Chess Data: {accuracy_pca * 100:.2f}%\n")
```

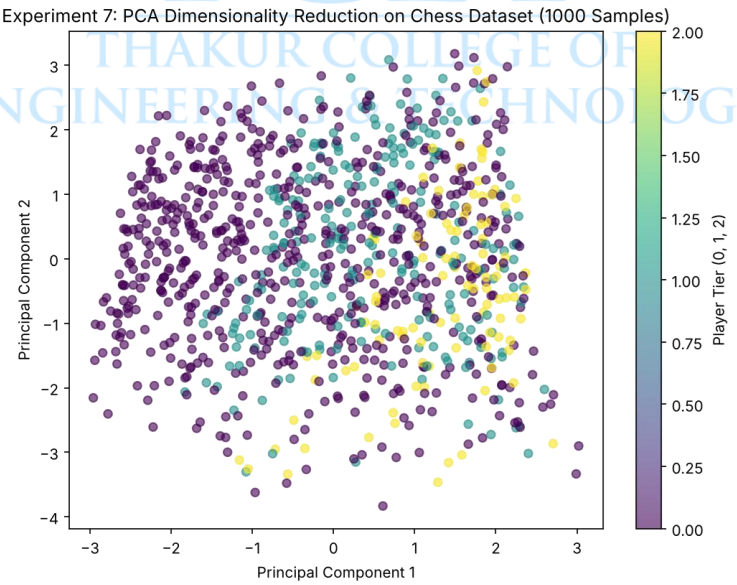
Output:

```
Accuracy of k-NN on PCA-reduced Chess Data: 59.50%
```

Code:

```
# Step 7: Visualize PCA Components
plt.figure(figsize=(8, 6))
scatter = plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y, cmap='viridis', alpha=0.6, s=30)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("Experiment 7: PCA Dimensionality Reduction on Chess Dataset (1000 Samples)")
plt.colorbar(scatter, label='Player Tier (0, 1, 2)')
plt.show()
```

Output:



Learning Outcomes: The student should have the ability to:
LO 7.1: Understand the principles of dimensionality reduction and the working of Principal Component Analysis (PCA).
LO 7.2: Standardize continuous feature variables and compute principal components to project high-dimensional data into a lower-dimensional space.
LO 7.3: Train and evaluate a K-Nearest Neighbors classifier on the dimensionally reduced feature set.
LO 7.4: Visualize and interpret the distribution of dataset classes along the top principal components.

Course Outcomes: Design, implement, and evaluate dimensionality reduction using Principal Component Analysis (PCA) by applying feature standardization, selecting an appropriate number of principal components, and assessing classifier performance and class distributions on the reduced feature space to solve real-world machine learning problems.

Conclusion:

For Faculty Use: -

Correction Parameters	Formative Assessment [40%]	Timely completion of Practical [40%]	Attendance / Learning Attitude [20%]	
Marks Obtained				

Experiment 8

Aim: Implementation of Reinforcement Learning

Learning Objective: To understand the basic concepts of reinforcement learning, represent a decision-making problem using states, actions, rewards and transitions, and implement the Q-learning algorithm in Python to train an agent that learns an effective path to a goal using the learned Q-table.

Tools: VS Code, I-Python Notebook.

Theory: -

1. Introduction

Machine learning is commonly divided into three paradigms: supervised learning, unsupervised learning and reinforcement learning. In supervised learning the model learns from labelled examples, and in unsupervised learning it discovers structure in unlabelled data. Reinforcement Learning (RL) is different from both, because the learner is not given a dataset at all. Instead, it learns by interacting with an environment and receiving feedback in the form of rewards or penalties.

Reinforcement learning is inspired by the way humans and animals learn through trial and error. A child learning to walk, or a chess player improving through practice, tries different actions, observes the consequences and gradually prefers the actions that lead to good outcomes. In the same way, an RL agent explores its environment, collects experience, and slowly builds a strategy that maximizes the total reward it receives over time.

2. Reinforcement Learning

Reinforcement Learning is a machine learning approach in which an agent learns to make a sequence of decisions by interacting with an environment. At each time step, the agent observes the current state, chooses an action, receives a reward, and moves to a new state. The objective of the agent is not to maximize the immediate reward, but to learn a policy that maximizes the cumulative reward collected over an entire episode.

This interaction is usually modelled as a Markov Decision Process (MDP), in which the next state depends only on the current state and the action taken, and not on the full history of earlier states. An MDP is described by a set of states, a set of actions, a transition rule that defines how actions change states, and a reward function that defines the feedback received for each transition.

The basic elements of Reinforcement Learning include:

- **Agent:** The learner or decision-making entity that takes actions.
- **Environment:** The system with which the agent interacts and which responds to its actions.
- **State (s):** The current situation of the agent within the environment.
- **Action (a):** A decision or move available to the agent in a given state.
- **Reward (r):** The numerical feedback received after taking an action.
- **Policy (π):** The strategy the agent uses to select actions in each state.
- **Q-value:** The expected cumulative reward of taking a particular action in a particular state and acting well afterwards.

3. Rewards, Returns and the Discount Factor

The reward signal tells the agent how good or bad an individual action was, but a good decision-maker must also consider the long-term consequences of its actions. The total reward the agent expects to collect from a state onwards is called the return. Since future rewards are less certain than immediate ones, they are multiplied by a discount factor γ (gamma), a value between 0 and 1.

$$G = r_1 + \gamma\, r_2 + \gamma^2\, r_3 + \ldots$$

A value of γ close to 0 makes the agent short-sighted and focused only on immediate rewards, while a value close to 1 makes it far-sighted and willing to accept small penalties now for a larger reward later. In this experiment γ = 0.95, so the agent values reaching the goal highly even when it is several moves away.

4. Q-Learning

Q-learning is a model-free, value-based reinforcement learning algorithm. It is called model-free because the agent does not need to know the transition rules or the reward function in advance; it learns purely from experience. It is called value-based because, instead of learning the policy directly, it learns a value for every state-action pair and then chooses the action with the highest value.

These values are stored in a Q-table, which has one row for each state and one column for each action. Initially all entries are zero. After every action, the agent updates the corresponding entry using the reward it received and the best estimated value of the next state, according to the Q-learning update equation:

$$Q(s, a) \leftarrow Q(s, a) + \alpha\, [\, r + \gamma \cdot \max Q(s', a') - Q(s, a)\,]$$

Here, s is the current state, a is the action taken, r is the reward received, s′ is the next state and a′ ranges over all possible actions in s′. The term $r + \gamma \cdot \max Q(s', a')$ is the new estimate of the value, and the difference between this estimate and the old value is called the temporal difference (TD) error. The learning rate α (alpha), between 0 and 1, controls how much of this error is used to update the old value. A high α makes the agent learn quickly from new experience, while a low α makes learning slower but more stable.

5. Exploration vs Exploitation

A key challenge in reinforcement learning is balancing exploration and exploitation. Exploitation means choosing the action that currently has the highest Q-value, using what the agent already knows. Exploration means trying a random action to discover whether something better exists. An agent that only exploits may get stuck with a poor strategy, while an agent that only explores never makes use of what it has learned.

Q-learning commonly handles this using the epsilon-greedy strategy. With probability ε (epsilon) the agent selects a random action, and with probability 1 − ε it selects the action with the highest Q-value. Training begins with ε = 1.0, so the agent explores completely at random. After every episode ε is multiplied by a decay factor (0.995 in this experiment) until it reaches a minimum value (0.05). As a result, the agent explores widely in the beginning and gradually shifts towards exploiting the knowledge it has gained.

6. Problem Formulation: 4 × 4 Grid World

In this experiment, the environment is a 4 × 4 grid containing 16 states numbered from 0 to 15. The agent starts at state 0 (top-left corner) and must reach the goal at state 15 (bottom-right corner). In each state, the

agent can choose one of four actions: Up, Down, Left or Right. If a move would take the agent outside the grid, it remains in the same cell.

The reward structure guides the agent towards an efficient path. Reaching the goal gives a reward of +10, hitting a wall and staying in place gives a penalty of −2, and every other move gives a penalty of −1. Because every extra step costs a penalty, the agent learns that the best strategy is to reach the goal using the fewest possible moves, which on a 4 × 4 grid is exactly six moves.

The Q-learning algorithm proceeds as follows:

- Initialize the Q-table with zeros.
- For each episode, place the agent at the starting state.
- Select an action using the epsilon-greedy strategy.
- Execute the action and observe the reward and the next state.
- Update the Q-value using the Q-learning equation.
- Move to the next state and repeat until the goal is reached or the maximum number of steps is completed.
- Decay epsilon after each episode.
- After training, choose the action with the highest Q-value at each state to follow the learned policy.

7. Applications of Reinforcement Learning

Reinforcement learning is used wherever a system must make a sequence of decisions under uncertainty and can learn from feedback. Some common applications include:

- Game playing, such as chess, Go and video games.
- Robotics, navigation and path planning.
- Autonomous vehicles and traffic signal control.
- Recommendation systems and online advertising.
- Resource management, scheduling and inventory control.
- Algorithmic trading and portfolio management.

8. Advantages

- Does not require labelled training data; learns directly from interaction.
- Q-learning is model-free and does not need knowledge of the environment's rules.
- Can learn long-term strategies by considering future rewards.
- Adapts to changing environments through continued learning.
- Simple to implement for problems with small, discrete state and action spaces.

9. Limitations

- Requires a large number of episodes to learn an effective policy.
- The Q-table grows with the number of states and actions, making it impractical for large or continuous problems.
- Performance depends heavily on the reward design and on hyperparameters such as α, γ and ε.
- Random exploration can be costly or unsafe in real-world environments.
- Learned Q-values may vary between runs because exploration is stochastic.

Code and Output:

Code:

```
import numpy as np

# 4 x 4 grid
n_states = 16
n_actions = 4

# Actions: 0=Up, 1=Down, 2=Left, 3=Right
q_table = np.zeros((n_states, n_actions))

alpha = 0.8          # learning rate
gamma = 0.95         # discount factor
epsilon = 1.0        # exploration probability
epsilon_min = 0.05
epsilon_decay = 0.995
episodes = 1000
max_steps = 50

start_state = 0
goal_state = 15

def next_state(state, action):
    row, col = divmod(state, 4)

    if action == 0 and row > 0:      # Up
        row -= 1
    elif action == 1 and row < 3:    # Down
        row += 1
    elif action == 2 and col > 0:    # Left
        col -= 1
    elif action == 3 and col < 3:    # Right
        col += 1

    return row * 4 + col

# Training
for episode in range(episodes):
    state = start_state

    for step in range(max_steps):
        if np.random.rand() < epsilon:
            action = np.random.randint(n_actions)
        else:
            action = np.argmax(q_table[state])

        new_state = next_state(state, action)

        if new_state == goal_state:
            reward = 10
        elif new_state == state:
            reward = -2
        else:
            reward = -1

        q_table[state, action] += alpha * (
            reward
            + gamma * np.max(q_table[new_state])
            - q_table[state, action]
        )

        state = new_state

        if state == goal_state:
            break

    epsilon = max(epsilon_min, epsilon *
epsilon_decay)

print("Learned Q-table:")
print(np.round(q_table, 2))
```

Output:

```
Learned Q-table:
[[ 1.05  3.21  1.05  3.21]
 [ 2.21  4.44  2.05  4.44]
 [ 3.44  5.72  3.21  5.72]
 [ 4.72  7.07  4.44  4.72]
 [ 2.05  4.44  2.21  4.44]
 [ 3.21  5.72  3.21  5.72]
 [ 4.44  7.07  4.44  7.07]
 [ 5.72  8.5   5.72  6.07]
 [ 3.21  5.72  3.44  5.72]
 [ 4.44  7.07  4.44  7.07]
```

```
[ 5.72  8.5   5.72  8.5 ]
[ 7.07 10.    7.07  7.5 ]
[ 4.44  4.72  4.72  7.07]
[ 5.72  6.07  5.72  8.5 ]
[ 7.07  7.5   7.07 10.  ]
[ 0.    0.    0.    0.  ]]
```

Code:

```
# Test the learned policy
state = start_state
path = [state]

for step in range(max_steps):
    action = np.argmax(q_table[state])
    state = next_state(state, action)
    path.append(state)
    if state == goal_state:
        break

print("\nLearned path:")
print(path)
print("Goal reached:", state == goal_state)
```

Output:

Learned path:
[0, 4, 8, 12, 13, 14, 15]
Goal reached: True

THAKUR EDUCATION TM
TCET
THAKUR COLLEGE OF
ENGINEERING & TECHNOLOGY

Learning Outcomes: The student should have the ability to:
LO 8.1: Understand the basic concepts of reinforcement learning and represent a decision-making problem using states, actions, rewards and transitions.
LO 8.2: Implement the Q-learning algorithm in Python using an epsilon-greedy strategy for balancing exploration and exploitation.
LO 8.3: Update Q-values using the learning rate and discount factor and interpret the learned Q-table.
LO 8.4: Use the learned Q-table to select an effective action sequence that moves the agent from the start state to the goal state.

Course Outcomes: Design, implement, and evaluate reinforcement learning agents using the Q-learning algorithm by formulating problems in terms of states, actions and rewards, tuning parameters such as the learning rate, discount factor and exploration rate, and interpreting the learned policy to solve sequential decision-making problems.

Conclusion:

THAKUR ZCT'S TM
TCET
THAKUR COLLEGE OF ENGINEERING & TECHNOLOGY

For Faculty Use: -

Correction Parameters	Formative Assessment [40%]	Timely completion of Practical [40%]	Attendance / Learning Attitude [20%]	
Marks Obtained				

Experiment 9

Aim: Hyperparameter Tuning using GridSearchCV / RandomizedSearchCV

Learning Objective: To understand the role of hyperparameters in machine learning models, perform hyperparameter tuning using GridSearchCV and RandomizedSearchCV with cross-validation, and compare the performance of the tuned models with a baseline model using appropriate evaluation metrics.

Tools: VS Code, I-Python Notebook.

Theory: -

1. Introduction

Every machine learning model has two kinds of settings. Model parameters, such as the weights of a linear model or the support vectors of an SVM, are learned automatically from the training data. Hyperparameters, on the other hand, are settings that must be chosen before training begins — for example, the regularization strength C and the kernel of an SVM, the number of neighbours K in KNN, or the depth of a decision tree. Hyperparameters control how the model learns, and they cannot be learned from the data in the same way as ordinary parameters.

The choice of hyperparameters has a strong effect on model performance. Poorly chosen values can cause underfitting, where the model is too simple to capture the patterns in the data, or overfitting, where the model memorizes the training data and performs badly on new data. Hyperparameter tuning is the process of systematically searching for the combination of hyperparameter values that gives the best performance on unseen data.

2. Hyperparameter Tuning

Manually trying different hyperparameter values is slow, error-prone and difficult to reproduce. Automated search methods solve this by defining a search space — the set of values each hyperparameter is allowed to take — and evaluating many candidate combinations using a consistent performance measure. The combination with the best score is then selected and used to train the final model.

A candidate combination should never be judged using the test set, because the test set must remain completely unseen until the final evaluation. If the test set were used to choose hyperparameters, the reported accuracy would be overly optimistic. Instead, each candidate is evaluated using cross-validation on the training data only.

The main components of hyperparameter tuning include:

- **Estimator:** The model or pipeline whose hyperparameters are being tuned, such as an SVM classifier.
- **Search Space:** The set of possible values (or distributions) for each hyperparameter.
- **Scoring Metric:** The measure used to compare candidates, such as accuracy, F1-score or RMSE.
- **Cross-Validation Strategy:** The method used to estimate how well each candidate generalizes, such as 5-fold cross-validation.
- **Best Estimator:** The model retrained on the full training set using the best hyperparameters found.

3. K-Fold Cross-Validation

Cross-validation provides a reliable estimate of how well a model with given hyperparameters will perform on unseen data. In K-fold cross-validation, the training data is divided into K equal parts called folds. The model is trained on K − 1 folds and validated on the remaining fold, and this process is repeated K times so that every fold is used for validation exactly once. The final score is the average of the K validation scores:

$$\text{CV Score} = (1 / K) \Sigma (i = 1 \text{ to } K) \text{ Score}_i$$

Averaging over several folds reduces the effect of a single lucky or unlucky split and gives a more stable comparison between candidates. For classification problems, stratified K-fold is used by default, so that each fold keeps the same proportion of classes as the full dataset. In this experiment, 5-fold cross-validation is used, which means every candidate combination is trained and evaluated five times.

4. GridSearchCV

GridSearchCV performs an exhaustive search over a manually specified grid of hyperparameter values. It evaluates every possible combination of the values provided, using cross-validation for each one, and returns the combination with the highest average score. For example, a grid with 4 values of C, 4 values of gamma and 2 kernels contains 4 × 4 × 2 = 32 combinations, and with 5-fold cross-validation this requires 160 model fits.

The main strength of GridSearchCV is that it is thorough: if the best combination lies within the grid, it is guaranteed to be found. However, the number of combinations grows multiplicatively with every hyperparameter and every additional value, which makes grid search very expensive for large search spaces. It also only tests the exact values listed, so a better value lying between two grid points can be missed.

5. RandomizedSearchCV

RandomizedSearchCV, instead of testing every combination, samples a fixed number of random combinations (controlled by the n_iter parameter) from the specified search space. Hyperparameters can be given either as lists of values or as continuous probability distributions. For parameters such as C and gamma, whose useful values span several orders of magnitude, a log-uniform distribution is commonly used so that small and large values are sampled equally often.

Because the number of candidates is fixed in advance, the cost of RandomizedSearchCV does not grow with the size of the search space. In practice, only a few hyperparameters usually have a large effect on performance, and random sampling explores many more distinct values of those important hyperparameters than a grid of the same size. As a result, randomized search often finds results that are as good as or better than grid search with fewer evaluations. Its limitation is that it is not exhaustive, so the very best combination is not guaranteed to be sampled.

6. Tuning a Support Vector Machine

In this experiment, a Support Vector Machine (SVM) classifier is tuned to predict the performance tier of chess players. The SVM is placed inside a Pipeline together with StandardScaler. Using a pipeline ensures that the scaler is fitted only on the training folds during cross-validation, which prevents information from the validation fold leaking into the training process. Hyperparameters of a step inside a pipeline are referred to using the step name followed by two underscores, such as svc__C.

The hyperparameters tuned for the SVM are:

- **C (Regularization Parameter):** Controls the trade-off between a smooth decision boundary and classifying training points correctly. A small C gives a wider, simpler margin, while a large C fits the training data more closely.
- **gamma:** Defines how far the influence of a single training point reaches for the RBF kernel. A small gamma gives smooth, broad decision boundaries, while a large gamma gives complex, tightly fitted boundaries.
- **kernel:** Determines the shape of the decision boundary — linear for straight-line boundaries or RBF for non-linear boundaries.

The tuning process includes:

- Preparing the dataset and creating the target labels.
- Splitting the data into training and testing sets.
- Training a baseline model with default hyperparameters.
- Defining the parameter grid and parameter distributions.
- Running GridSearchCV and RandomizedSearchCV with 5-fold cross-validation.
- Evaluating the best models on the test set and comparing the results.

7. Applications of Hyperparameter Tuning

Hyperparameter tuning is a standard step in almost every practical machine learning workflow. Some common applications include:

- Selecting C, gamma and kernel for Support Vector Machines.
- Choosing the number of neighbours and distance metric for KNN.
- Tuning tree depth and number of estimators in Random Forests and Gradient Boosting.
- Selecting learning rate, batch size and layer sizes in neural networks.
- Choosing regularization strength in linear and logistic regression.
- Building automated machine learning (AutoML) pipelines.

8. Advantages

- Improves model accuracy and generalization compared to default settings.
- Replaces manual trial-and-error with a systematic and reproducible process.
- Uses cross-validation, giving reliable estimates and reducing overfitting to one split.
- GridSearchCV is exhaustive within the given grid.
- RandomizedSearchCV handles large and continuous search spaces efficiently.

9. Limitations

- Computationally expensive, since every candidate is trained K times.
- The cost of GridSearchCV grows rapidly as more hyperparameters and values are added.
- RandomizedSearchCV may miss the best combination because it is not exhaustive.
- Results depend on how well the search space is defined.
- Over-tuning on a small dataset can still overfit the cross-validation folds.

Code and Output:

Code:

```
#Importing the Libraries
import pandas as pd
import numpy as np
import time
import matplotlib.pyplot as plt
from scipy.stats import loguniform
from sklearn.model_selection import
train_test_split, GridSearchCV,
RandomizedSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# Step 1: Generate 1000 entries of Chess Dataset
np.random.seed(42)
n_samples = 1000
df = pd.DataFrame({
    'Elo_Rating': np.random.randint(2400, 2880,
n_samples),
    'Rapid_Rating': np.random.randint(2300, 2850,
n_samples),
    'Blitz_Rating': np.random.randint(2300, 2900,
n_samples),
    'Age': np.random.randint(16, 48, n_samples),
    'Wins': np.random.randint(0, 15, n_samples),
    'Draws': np.random.randint(2, 20, n_samples),
    'Losses': np.random.randint(0, 10, n_samples),
})
df['Total_Games'] = df['Wins'] + df['Draws'] +
df['Losses']
df['Win_Ratio'] = df['Wins'] / df['Total_Games']

# Step 2: Create Target Label (3 Player Tiers)
def assign_tier(row):
    if row['Elo_Rating'] >= 2760 and
row['Win_Ratio'] >= 0.35:
        return 2
    elif row['Elo_Rating'] >= 2650 and
row['Win_Ratio'] >= 0.20:
        return 1
    else:
        return 0

df['Tier'] = df.apply(assign_tier, axis=1)
df.head()
```

Output:

	Elo_Rating	Rapid_Rating	Blitz_Rating	Age	Wins	Draws	Losses	Total_Games	Win_Ratio	Tier
0	2502	2300	2680	32	1	9	7	17	0.0588	0
1	2835	2350	2401	24	7	12	5	24	0.2917	1
2	2748	2745	2793	45	12	2	2	16	0.7500	1
3	2670	2748	2318	27	8	7	4	19	0.4211	1
4	2506	2787	2753	18	12	17	1	30	0.4000	0

Code:

```
# Step 3: Split Features and Target
X = df.drop('Tier', axis=1)
y = df['Tier']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42,
stratify=y
)

# Step 4: Baseline Model with Default
Hyperparameters
pipe = Pipeline([
    ('scaler', StandardScaler()),
```

```
    ('svc', SVC(random_state=42))
])
pipe.fit(X_train, y_train)
base_acc = accuracy_score(y_test,
pipe.predict(X_test))
print("Default Parameters: C=1.0, gamma='scale',
kernel='rbf'")
print(f'Baseline SVM Test Accuracy: {base_acc *
100:.2f}%")
```

Output:

```
Default Parameters: C=1.0, gamma='scale', kernel='rbf'
Baseline SVM Test Accuracy: 89.00%
```

Code:

```
# Step 5: Hyperparameter Tuning using
GridSearchCV
param_grid = {
    'svc__C': [0.1, 1, 10, 100],
    'svc__gamma': ['scale', 0.01, 0.1, 1],
    'svc__kernel': ['linear', 'rbf']
}
grid = GridSearchCV(pipe, param_grid, cv=5,
                    scoring='accuracy', n_jobs=-1)
start = time.time()
grid.fit(X_train, y_train)
grid_time = time.time() - start
grid_acc = accuracy_score(y_test,
grid.predict(X_test))

print("GridSearchCV Results")
print("Combinations Tried:",
len(grid.cv_results_['params']))
print("Best Parameters:", grid.best_params_)
print(f'Best CV Accuracy: {grid.best_score_ *
100:.2f}%")
print(f"Test Accuracy: {grid_acc * 100:.2f}%")
print(f"Time Taken: {grid_time:.2f} seconds")
```

Output:

```
GridSearchCV Results
Combinations Tried: 32
Best Parameters: {'svc__C': 10, 'svc__gamma': 0.1, 'svc__kernel': 'rbf'}
Best CV Accuracy: 92.38%
Test Accuracy: 92.00%
Time Taken: 4.15 seconds
```

Code:

```
# Step 6: Hyperparameter Tuning using
RandomizedSearchCV
param_dist = {
    'svc__C': loguniform(1e-2, 1e3),
    'svc__gamma': loguniform(1e-3, 1e1),
    'svc__kernel': ['linear', 'rbf']
}
rand = RandomizedSearchCV(pipe, param_dist,
n_iter=20, cv=5,
                          scoring='accuracy',
random_state=42,
                          n_jobs=-1)
start = time.time()
rand.fit(X_train, y_train)
rand_time = time.time() - start
rand_acc = accuracy_score(y_test,
rand.predict(X_test))

best = rand.best_params_
print("RandomizedSearchCV Results")
print("Combinations Tried:",
len(rand.cv_results_['params']))
print(f"Best Parameters: C={best['svc__C']:.4f}, "
      f"gamma={best['svc__gamma']:.4f},
kernel={best['svc__kernel']}")
print(f'Best CV Accuracy: {rand.best_score_ *
100:.2f}%")
print(f"Test Accuracy: {rand_acc * 100:.2f}%")
```

print(f"Time Taken: {rand_time:.2f} seconds")

Output:

RandomizedSearchCV Results
Combinations Tried: 20
Best Parameters: C=739.2266, gamma=0.0085, kernel=rbf
Best CV Accuracy: 94.00%
Test Accuracy: 92.00%
Time Taken: 4.90 seconds

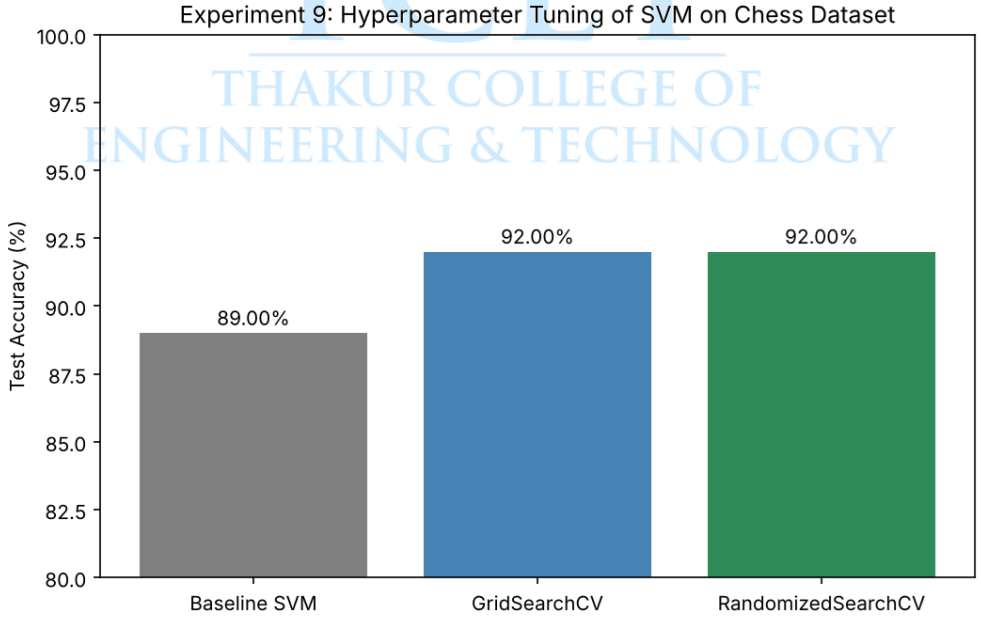
Code:

```
# Step 7: Compare and Visualize Results
results = pd.DataFrame({
    'Model': ['Baseline SVM', 'GridSearchCV', 'RandomizedSearchCV'],
    'Test Accuracy (%)': [base_acc * 100, grid_acc * 100, rand_acc * 100]
})
print(results.round(2))

plt.figure(figsize=(8, 5))
bars = plt.bar(results['Model'], results['Test Accuracy (%)'],
               color=['gray', 'steelblue', 'seagreen'])
for bar in bars:
    plt.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.3,
             f"{bar.get_height():.2f}%", ha='center')
plt.ylim(80, 100)
plt.ylabel("Test Accuracy (%)")
plt.title("Experiment 9: Hyperparameter Tuning of SVM on Chess Dataset")
plt.show()
```

Output:

	Model	Test Accuracy (%)
0	Baseline SVM	89.0
1	GridSearchCV	92.0
2	RandomizedSearchCV	92.0



Learning Outcomes: The student should have the ability to:
LO 9.1: Understand the difference between model parameters and hyperparameters and the importance of hyperparameter tuning.
LO 9.2: Apply K-fold cross-validation to reliably evaluate candidate hyperparameter combinations.
LO 9.3: Implement hyperparameter tuning using GridSearchCV and RandomizedSearchCV with a preprocessing pipeline.
LO 9.4: Compare tuned models with a baseline model and analyze the trade-off between search thoroughness and computational cost.

Course Outcomes: Design, implement, and evaluate optimized machine learning models by applying hyperparameter tuning techniques such as GridSearchCV and RandomizedSearchCV with cross-validation and preprocessing pipelines, and comparing model performance against a baseline to select the best model for real-world machine learning problems.

Conclusion:

For Faculty Use: -

Correction Parameters	Formative Assessment [40%]	Timely completion of Practical [40%]	Attendance / Learning Attitude [20%]	
Marks Obtained				